



COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

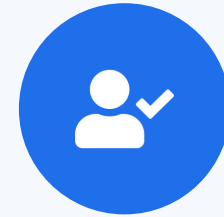


COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.



**Minimum 75%
attendance required**

About the Trainer



Mohan Pothula

Kubernetes & Cloud-Native Trainer



Role

Principal Kubernetes & DevOps trainer at Tertiary Infotech Academy.



Qualifications

Certified Kubernetes Application Developer (CKAD) · CKA · Docker Certified Associate · AWS Solutions Architect.



Expertise

Kubernetes · Docker · CI/CD · Cloud-native platform engineering.



Experience

20+ years across DBS Bank, SingTel, Mediacorp and SPH Media.

About the Trainer



[Trainer Name]

[Title / Role]



Qualifications:



Expertise:



Experience:



Contact / Profile:

Let's Know Each Other

Take a minute to introduce yourself to the class:



Your role

Your name and organisation / role.



Your experience

Any experience with Docker, Kubernetes or the cloud.



Your goal

Whether you are targeting the CKAD exam or production skills.

Ground Rules



Phones on silent

Set your phone to silent mode.



Participate actively

No question is too small — ask freely.



Mutual respect

One conversation at a time.



Be punctual

Return from breaks on time.



Step out quietly

For calls or short breaks.



75% attendance

Required for funding eligibility.

Lesson Plan — 3½ Days, 8 hours/day

1

Day 1 · Domain 1

Application Design & Build

- Container images & multi-stage builds (Labs 1–2)
- Pods, Jobs & CronJobs (Labs 3–5)
- Multi-container Pods & Volumes (Labs 6–8)

2

Day 2 · Domain 2

Application Deployment

- Deployments, ReplicaSets & rollouts (Labs 9–10)
- Blue/green & canary (Labs 11–12)
- Helm, Kustomize, Daemon/StatefulSet (Labs 13–15)

3

Day 3 · Domains 3 & 4

Observability + Config & Security

- Probes, logging, metrics, debug (Labs 16–20)
- ConfigMaps, Secrets, SecurityContext (Labs 21–23)
- ServiceAccounts, RBAC, Quotas (Labs 24–26)

4

Day 4 (½) · Domain 5

Services & Networking + Assessment

- Services & Service DNS (Labs 27–28)
- Ingress with TLS & NetworkPolicy (Labs 29–30)
- 1:00 PM mock-exam practice, then final assessment from 2:00 PM

Daily timing: 9:00am – 6:00pm · 1-hour lunch · short tea breaks within each day

Learning Outcomes

- 1 **LO1** Build and optimise container images with single- and multi-stage Dockerfiles.
- 2 **LO2** Create and manage Pods imperatively and declaratively with kubectl and --dry-run.
- 3 **LO3** Run batch workloads with Jobs and scheduled workloads with CronJobs.
- 4 **LO4** Design multi-container Pods using the sidecar and init-container patterns.
- 5 **LO5** Persist and share Pod data with emptyDir, tmpfs and hostPath volumes.
- 6 **LO6** Deploy, scale, observe, secure and network applications across the five CKAD domains.

Assessment



Final Assessment

- Written / MCQ assessment on the LMS
- Mock-exam practice from 1:00 PM; final assessment from 2:00 PM
- Covers all five CKAD domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding



Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the written / MCQ assessment.

Briefing for Assessment



Clear your space

Keep only approved materials on the table.



No recording

No photos of the assessment.



Work individually

No discussion during the assessment.



Use the LMS

The assessment is delivered online.



Open book

Slides, Learner Guide & kubernetes.io are allowed.



Time's up

Submit when time is up.

DAY 1

Application Design and Build

Images · Pods · Jobs · CronJobs · Multi-container · Volumes (CKAD Domain 1 — 20%)



TOPIC 1

Container Images

Dockerfiles, image layers & multi-stage builds

Why Container Images?

- An image is a read-only blueprint that packages your app with all its dependencies.
- A container is a running instance of an image — it starts in milliseconds.
- Kubernetes never builds images; it pulls them from a registry by name:tag and runs them.
- CKAD expects you to read, write and fix Dockerfiles under time pressure.

ONE INSTRUCTION = ONE LAYER

Anatomy of a Dockerfile

Base & setup

- FROM — pick a small, pinned base
- WORKDIR — set the working dir
- COPY / ADD — bring files in

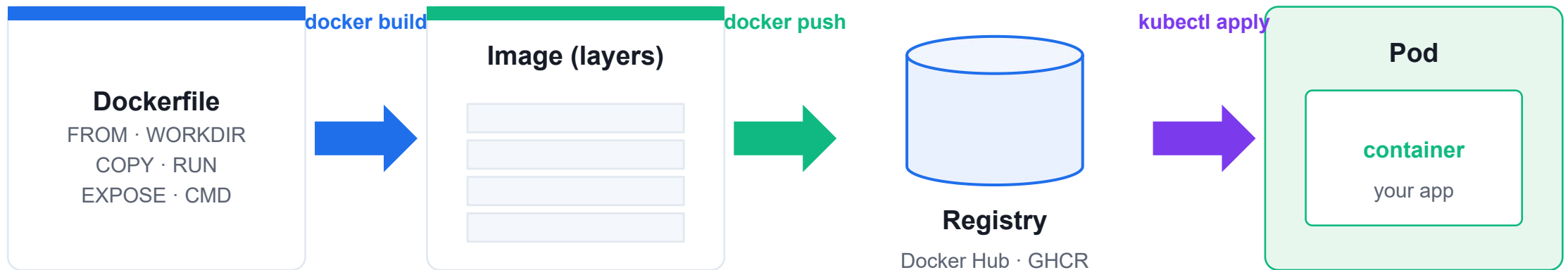
Build steps

- RUN — install dependencies
- Each line = one cached layer
- Order steps for cache reuse

Runtime

- EXPOSE — document the port
- ENV — runtime configuration
- CMD / ENTRYPOINT — what runs

From Image to a Running Pod



Kubernetes never builds images — it **pulls them from a registry by name:tag** and runs them inside a Pod.

Single-Stage vs Multi-Stage Builds

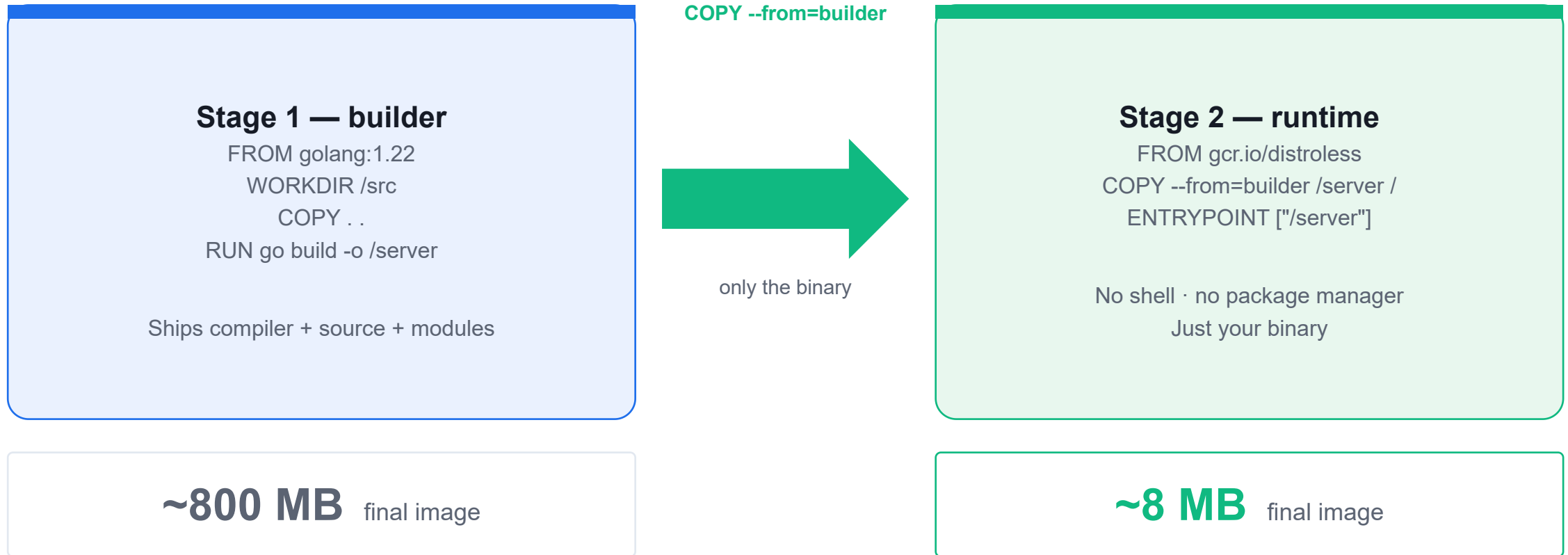
Single-stage

- One FROM — build + runtime together
- Ships the whole toolchain
- Large image (~800 MB for Go)
- Bigger attack surface

Multi-stage

- Multiple FROM stages with AS <name>
- COPY --from=builder copies only the binary
- Tiny image (~8 MB, distroless)
- No shell, no package manager

Multi-Stage Build — Shrink the Image



Build a Container Image with Docker

LAB 1

Container images

Write a Dockerfile from scratch, build a tagged image, run it locally and inspect its layers. CKAD expects you to read and write Dockerfiles confidently — you may be asked to fix a broken one or produce one under time pressure.

You'll build

Write a Dockerfile, build and tag an image, run it, then inspect its layers with docker history.

Key commands: `FROM python:3.12-slim` · `» `EXPOSE`` · `docker build` · `docker run` · `docker history`

Build a Container Image with Docker

Step 1 — Write the Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
EXPOSE 8080
CMD ["python", "app.py"]
```

Note EXPOSE is documentation only — it does not publish the port. -p host:container is what actually opens it.

Step 2 — Build and tag the image

```
docker build -t ckad/hello:1.0 .
docker images ckad/hello
```

Build a Container Image with Docker (cont.)

Step 3 — Run and test the container

```
docker run -d --name hello -p 8080:8080 ckad/hello:1.0  
curl http://localhost:8080
```

Step 4 — Inspect the image layers

```
docker history ckad/hello:1.0  
docker rm -f hello && docker rmi ckad/hello:1.0
```

Build a Container Image with Docker

✓ Test it

`curl http://localhost:8080` returns hello from CKAD 2026 lab 1, and docker history shows a distinct layer for each Dockerfile instruction.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Multi-Stage Dockerfile

LAB 2

Container images

Separate the build stage from the runtime stage to produce images that are 10x smaller and contain no compiler toolchain. Multi-stage builds are a CKAD staple — you must be able to write one from scratch and explain why it shrinks the image.

You'll build

Build a single-stage baseline, then a multi-stage image with COPY --from and distroless, and compare sizes.

Key commands: FROM golang:1.22 · docker build · » Only

Multi-Stage Dockerfile

Step 1 — Single-stage baseline (large image)

```
FROM golang:1.22
WORKDIR /src
COPY main.go .
RUN go mod init demo && go build -o app main.go
CMD ["/app"]
```

Step 2 — Multi-stage Dockerfile (small image)

```
# Stage 1: compile
FROM golang:1.22 AS builder
WORKDIR /src
COPY main.go .
RUN go mod init demo && CGO_ENABLED=0 go build -o /out/app main.go

# Stage 2: runtime only
FROM gcr.io/distroless/static-debian12
COPY --from=builder /out/app /app
ENTRYPOINT ["/app"]
```


Multi-Stage Dockerfile (cont.)

Step 3 — Build both and compare sizes

```
docker build -f Dockerfile.single -t demo:single .  
docker build -f Dockerfile.multi -t demo:multi .  
docker images | grep demo          # ~800MB vs ~8MB
```

Note Only the last FROM stage ends up in the final image. CGO_ENABLED=0 produces a static binary that runs in distroless/scratch — no shell, smaller attack surface.

Multi-Stage Dockerfile

✓ Test it

docker images | grep demo shows demo:multi roughly 100x smaller than demo:single, and demo:multi still serves hello from multi-stage build.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 2

Pods

The smallest deployable unit & the exam-speed kubectl workflow

What is a Pod?

- A Pod is the smallest deployable unit — one or more containers sharing network & storage.
- Containers in a Pod share an IP and localhost; they can share volumes.
- Create Pods imperatively (`kubectl run`) or declaratively (`kubectl apply -f`).
- Pods are ephemeral — in production you run them via higher-level controllers (Day 2).

The CKAD kubectl Workflow

Generate

- kubectl run / create
- with --dry-run=client -o yaml
- Scaffold a manifest fast

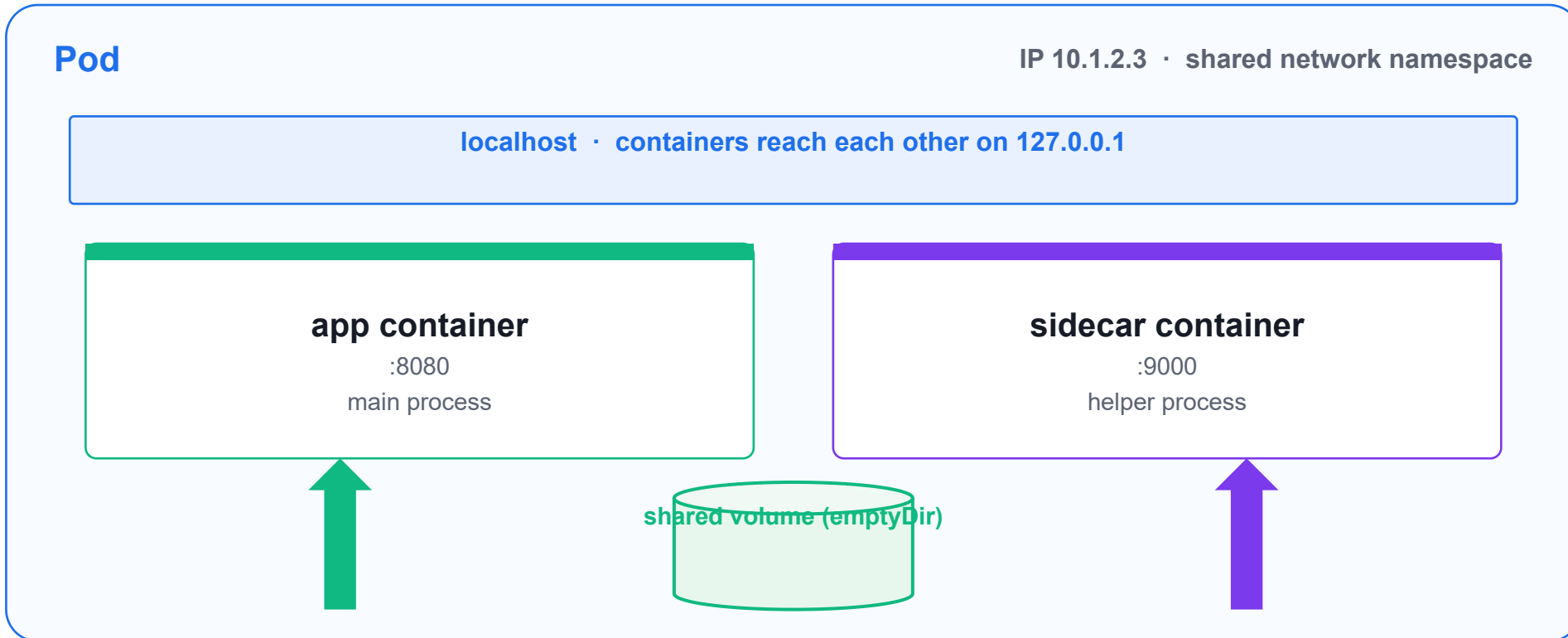
Edit

- Redirect to a .yaml file
- Tweak labels, image, command
- Version-controllable

Apply

- kubectl apply -f file.yaml
- Idempotent & repeatable
- kubectl describe to verify

Anatomy of a Pod



A Pod groups containers that must run together — they **share one IP, localhost and volumes**, and live and die as a unit.

Exam-Speed kubectl (kubernetes.io quick reference)

- `alias k=kubectl` — set this first; saves keystrokes on every command.
- `export do="--dry-run=client -o yaml"` — scaffold any manifest without creating it.
- `source <(kubectl completion bash); complete -o default -F __start_kubectl k` — tab-completion.
- `kubectl get -o wide / -o yaml / -o jsonpath` · `kubectl describe` · `kubectl explain` — inspect & learn.

Create and Manage Pods

LAB 3

Pods

The Pod is the smallest schedulable unit in Kubernetes. Create Pods imperatively, generate YAML with `--dry-run`, edit live manifests, and use the exam-critical `$do` alias that saves 30+ seconds per question.

You'll build

Set exam-speed aliases, run Pods imperatively, scaffold YAML with `$do`, then apply and inspect.

Key commands: `alias k=kubectl · k run · k describe · » `--restart=Never``

Create and Manage Pods

Step 1 — Set exam-speed aliases (run these first on exam day)

```
alias k=kubectl  
export do="--dry-run=client -o yaml"
```

Step 2 — Create a Pod imperatively

```
k run web --image=nginx:1.25 --port=80  
k get pod web -o wide          # shows node and Pod IP
```

Step 3 — Generate a manifest without creating it

```
k run web2 --image=nginx:1.25 --port=80 $do > web2.yaml  
k apply -f web2.yaml  
k get pod web2 --show-labels
```

Create and Manage Pods (cont.)

Step 4 — Inspect a running Pod

```
k describe pod web
k get pod web -o jsonpath='{.status.podIP}'; echo
```

Step 5 — Override the container command

```
k run sleeper --image=busybox --restart=Never -- sh -c 'sleep 3600'
k delete pod web web2 sleeper --force --grace-period=0
```

Note `--restart=Never` creates a bare Pod (not a Deployment). Everything after `--` becomes the container command. `--force --grace-period=0` skips the 30s grace period.

Create and Manage Pods

✓ Test it

`k get pod web2 --show-labels` shows the `tier=frontend` label, and the `--dry-run=client -o yaml` workflow produces a valid manifest you can apply.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Lunch Break

1 hour



TOPIC 3

Jobs & CronJobs

Run-to-completion & scheduled batch workloads

Batch Workloads in Kubernetes

- A Job runs Pods until a set number of successful completions, then stops — it does not restart on success.
- A CronJob creates Jobs on a schedule using standard cron syntax (*/* * * * * = every minute).
- Job Pod templates must use restartPolicy: Never or OnFailure — never Always.
- Trigger a CronJob now with: `kubectl create job --from=cronjob/<name> <job-name>`.

Key Fields to Memorise

Job

- completions — successes needed
- parallelism — Pods at once
- backoffLimit — failures allowed

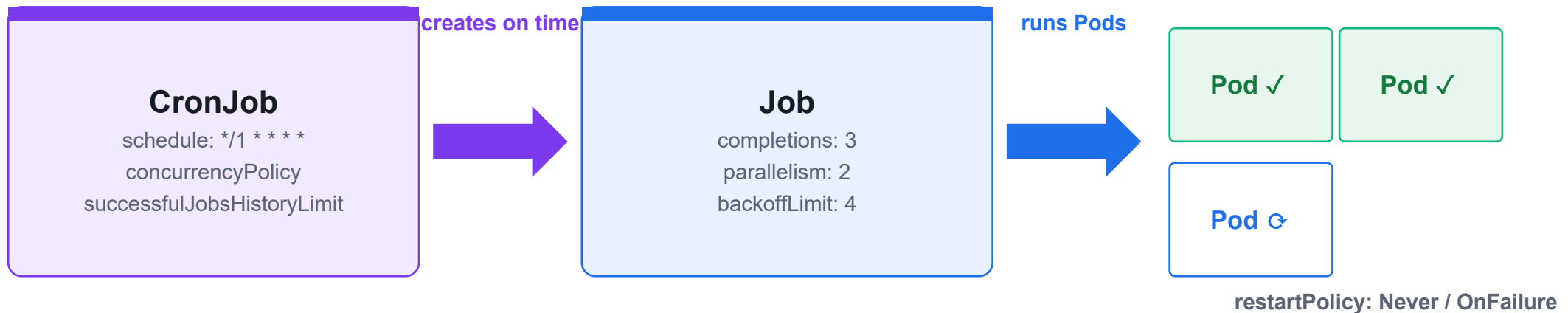
Job (timing)

- activeDeadlineSeconds
- Hard wall-clock ceiling
- Overrides backoffLimit

CronJob

- schedule + timeZone
- concurrencyPolicy:
Allow/Forbid/Replace
- successful/failedJobsHistoryLimit

How CronJob, Job & Pods Relate



A CronJob stamps out a Job on a schedule; the Job runs Pods until **completions succeed**, then stops. Jobs never use restartPolicy: Always.

Jobs — Run-to-Completion Workloads

LAB 4

Batch workloads

A Job runs Pods until a required number of successful completions is reached, then stops. CKAD tests completions, parallelism, backoffLimit and activeDeadlineSeconds in almost every sitting — you must write a Job manifest from memory.

You'll build

Create a one-shot Job, a parallel Job with completions, and explore backoffLimit and activeDeadlineSeconds.

Key commands: `k create · apiVersion: batch/v1 · k apply · » `restartPolicy:`

Jobs — Run-to-Completion Workloads

Step 1 — One-shot Job (imperative)

```
k create job hello --image=busybox -- echo "hello CKAD 2026"  
k logs -l job-name=hello
```

Jobs — Run-to-Completion Workloads (cont.)

Step 2 — Parallel Job with multiple completions

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-parallel
spec:
  completions: 5
  parallelism: 2
  backoffLimit: 4
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: pi
        image: perl:5.34
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(50)"]
```

Jobs — Run-to-Completion Workloads (cont.)

Step 3 — Apply and watch completions climb

```
k apply -f parallel.yaml  
k get job pi-parallel -w      # COMPLETIONS ticks 0/5 → 5/5
```

Note restartPolicy: Never is mandatory in Job Pod templates. activeDeadlineSeconds is a hard wall-clock ceiling — it overrides backoffLimit.

Jobs — Run-to-Completion Workloads

✓ Test it

k get job pi-parallel reaches 5/5 completions with no more than two Pods running simultaneously during the run.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

CronJobs — Scheduled Workloads

LAB 5

Batch workloads

A CronJob creates Jobs on a time-based schedule using standard cron syntax. CKAD tests concurrencyPolicy, timeZone, startingDeadlineSeconds, history limits, manual triggers and suspend/resume — these fields appear verbatim in exam questions.

You'll build

Create a CronJob imperatively and declaratively, suspend/resume it, then trigger it manually.

Key commands: `k create · apiVersion: batch/v1 · k patch · » `concurrencyPolicy`:`

CronJobs — Scheduled Workloads

Step 1 — Create a CronJob imperatively

```
k create cronjob date-printer --image=busybox \  
  --schedule="*/1 * * * *" -- sh -c "date; echo from cronjob"  
k get cronjob date-printer
```


CronJobs — Scheduled Workloads (cont.)

Step 2 — Declarative CronJob with all key fields

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: report
spec:
  schedule: "*/2 * * * *"
  timeZone: "Asia/Singapore"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 2
  startingDeadlineSeconds: 30
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: busybox
              command: ["sh", "-c", "echo report at $(date)"]
```

CronJobs — Scheduled Workloads (cont.)

Step 3 — Suspend, resume and trigger manually

```
k patch cronjob report -p '{"spec":{"suspend":true}}'      # pause
k patch cronjob report -p '{"spec":{"suspend":false}}'     # resume
k create job --from=cronjob/report report-manual           # run now
```

Note concurrencyPolicy: Allow (default) / Forbid / Replace. timeZone is a newer field — always set it. create job --from=cronjob/<name> runs the workload immediately.

CronJobs — Scheduled Workloads

✓ Test it

k get cronjob report shows SUSPEND True after the patch, and k create job --from=cronjob/report report-manual produces an immediate one-off run.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 4

Multi-Container Pods

Sidecar & init-container design patterns

Why More Than One Container?

- Containers in a Pod share the network namespace and can share volumes (emptyDir is the usual glue).
- Sidecar — a helper alongside the main app (log shipper, proxy) sharing a volume.
- Init container — runs to completion before main containers (seed a volume, wait for a service).
- Always target a specific container with `kubectl logs -c <name>` and `kubectl exec -c <name>`.

Container Patterns

Sidecar

- Runs beside the main app
- Shares an emptyDir volume
- Log shipping / proxy / sync

Init container

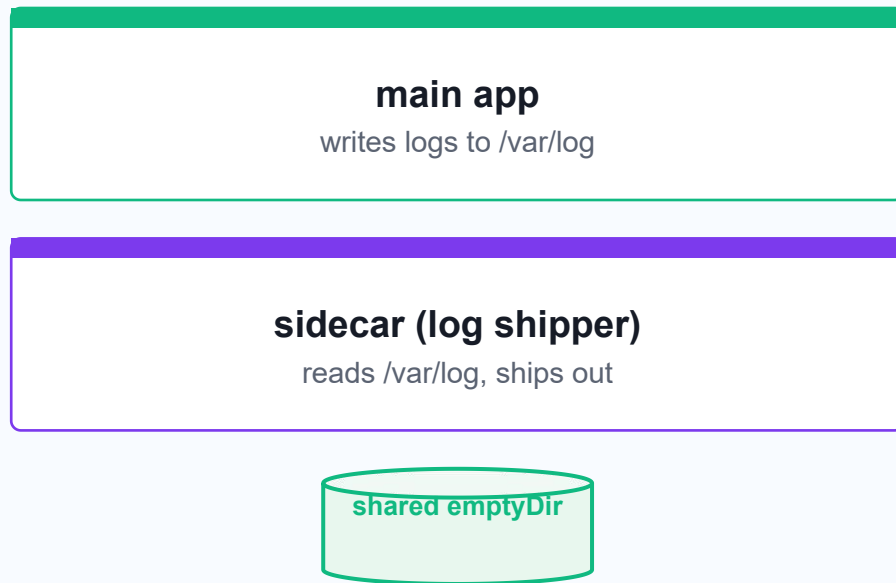
- Runs first, to completion
- Pod shows Init:N/M
- Seed volume · wait for DNS

Native sidecar

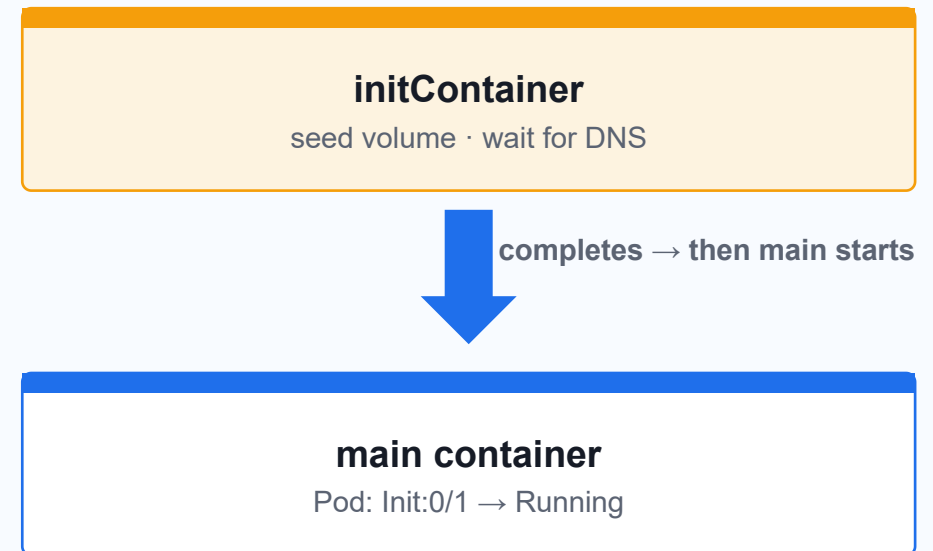
- initContainer + restartPolicy: Always
- Kubernetes 1.29+
- Runs for the Pod's lifetime

Multi-Container Pod Patterns

Sidecar — run side by side



Init container — run first, to completion



Sidecars run **alongside** the app; init containers run **before** it. Target one with `kubectl logs/exec -c <name>`.

Multi-Container Pods — Sidecar Pattern

LAB 6

Multi-container Pods

Containers in the same Pod share a network namespace and can share volumes. CKAD tests the sidecar pattern (a helper reads/writes a shared volume) and the native sidecar feature (Kubernetes 1.29+). You must exec into and read logs from each container.

You'll build

Run a sidecar that tails a shared emptyDir log, read each container's logs, then build a native sidecar.

Key commands: `apiVersion: v1 · k logs · initContainers: · »` With

Multi-Container Pods — Sidecar Pattern

Step 1 — Sidecar sharing a log file via emptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-sidecar
spec:
  volumes:
    - name: logs
      emptyDir: {}
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "while true; do date >> /var/log/app.log; sleep 2; done"]
      volumeMounts:
        - {name: logs, mountPath: /var/log}
    - name: log-shipper
      image: busybox
      command: ["sh", "-c", "tail -F /var/log/app.log"]
      volumeMounts:
        - {name: logs, mountPath: /var/log}
```

Multi-Container Pods — Sidecar Pattern (cont.)

Step 2 — Read logs from each container separately

```
k logs app-with-sidecar -c app | head -5
k logs app-with-sidecar -c log-shipper | head -5
k exec -it app-with-sidecar -c app -- sh -c 'wc -l /var/log/app.log'
```

Step 3 — Native sidecar container (Kubernetes 1.29+)

```
initContainers:
- name: log-collector
  image: busybox
  restartPolicy: Always
  command: ["sh", "-c", "while true; do echo alive; sleep 5; done"]
```

Note With multiple containers in a Pod, always specify `-c <name>` for logs and exec. `emptyDir` is the standard glue volume between sidecar and main containers.

Multi-Container Pods — Sidecar Pattern

✓ Test it

k logs app-with-sidecar -c log-shipper shows the same lines app is writing, proving both containers share the emptyDir volume.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Init Containers

LAB 7

Multi-container Pods

Init containers run in order, to completion, before any main container starts. Use them to seed shared volumes, wait for upstream services, or do one-time setup. CKAD asks you to write init containers and read Init:N/M Pod status correctly.

You'll build

Seed an nginx web root with an init container, then add an init container that waits for a Service via DNS.

Key commands: `apiVersion: v1 · k exec · initContainers: · »` The

Init Containers

Step 1 — Init container that seeds the web root

```
apiVersion: v1
kind: Pod
metadata:
  name: web-init
spec:
  volumes:
    - name: html
      emptyDir: {}
  initContainers:
    - name: seed
      image: busybox
      command: ["sh", "-c", "echo '<h1>Seeded by init</h1>' > /work/index.html"]
      volumeMounts:
        - {name: html, mountPath: /work}
  containers:
    - name: web
      image: nginx:1.25
      volumeMounts:
        - {name: html, mountPath: /usr/share/nginx/html}
```

Init Containers (cont.)

Step 2 — Verify the seeded content

```
k exec web-init -- cat /usr/share/nginx/html/index.html
```

Step 3 — Init container that waits for a Service

```
initContainers:  
- name: wait-for-db  
  image: busybox  
  command: ["sh", "-c", "until nslookup db.default.svc.cluster.local; do sleep 2; done"]
```

Note The Pod stays in Init:0/1 until DNS resolves. Creating the db Service unblocks it. If an init container fails it is restarted per the Pod's restartPolicy.

Init Containers

✓ Test it

`k exec web-init -- cat /usr/share/nginx/html/index.html` returns the seeded heading, proving the init container populated the volume before nginx started.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 5

Volumes

Persist & share Pod data: emptyDir, tmpfs & hostPath

Pod Storage Basics

- Anything written to the container filesystem is lost when the container restarts.
- Volumes are declared under `spec.volumes` and mounted via `spec.containers[].volumeMounts`.
- A volume can be shared by every container in the Pod — that is how sidecars exchange data.
- Pod-scoped volumes live and die with the Pod; durable storage (PV/PVC) comes later in the course.

Volume Types in Domain 1

emptyDir: {}

- Created with the Pod
- Shared by all containers
- Deleted with the Pod

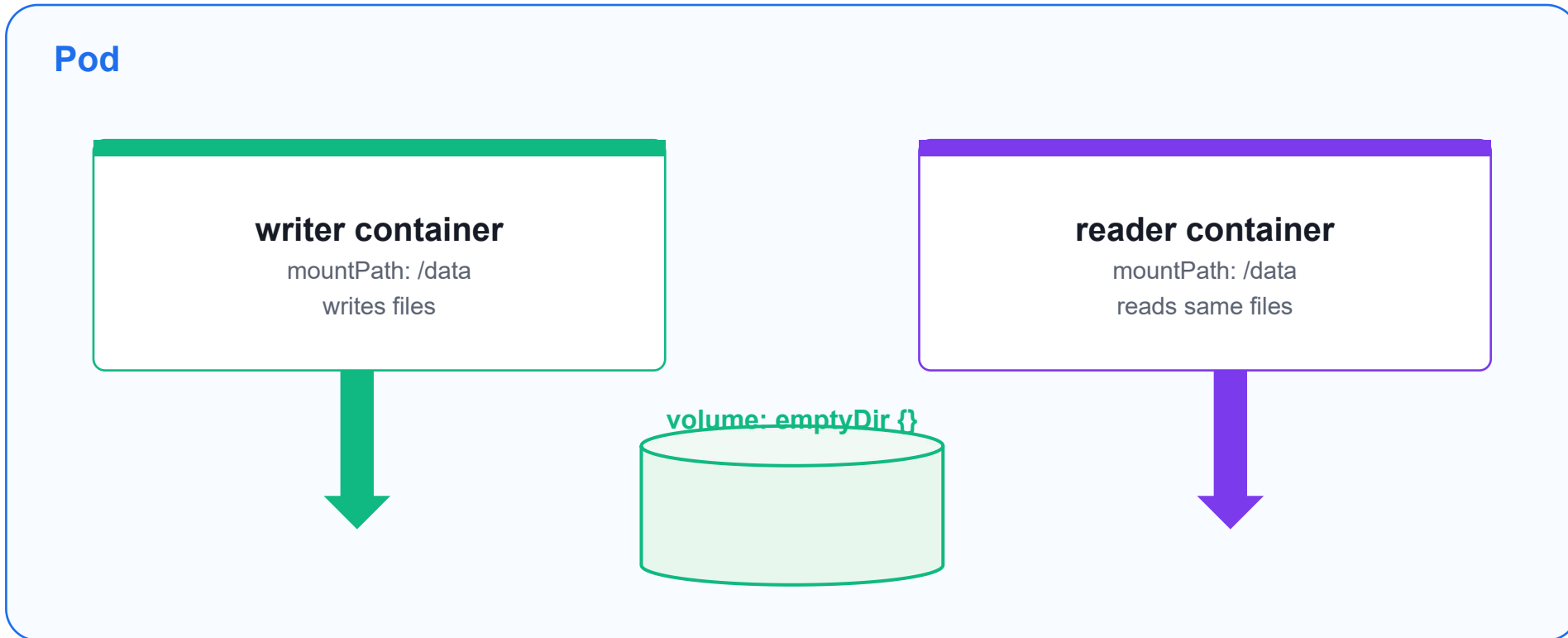
emptyDir: Memory

- tmpfs — backed by RAM
- Fast scratch space
- Counts to memory limit

hostPath

- Mounts a node directory
- DaemonSets / node tools only
- Security risk in multi-tenant

Sharing Data with a Volume



Declare it under **spec.volumes**, mount it in each container via **volumeMounts** — created with the Pod, deleted with the Pod.

Volumes — emptyDir and hostPath

LAB 8

Volumes

Containers are ephemeral — data written to the container filesystem is lost on restart. Volumes survive restarts and can be shared between containers. CKAD tests emptyDir, emptyDir.medium: Memory and hostPath, plus mounting strategies.

You'll build

Share an emptyDir between two containers, mount a RAM-backed tmpfs volume, then mount a node directory with hostPath.

Key commands: `volumes: · » Volumes`

Volumes — emptyDir and hostPath

Step 1 — emptyDir shared between two containers

```
volumes:
- name: shared
  emptyDir: {}
containers:
- name: writer
  image: busybox
  command: ["sh", "-c", "echo hello-shared > /data/msg.txt; sleep 3600"]
  volumeMounts: [{name: shared, mountPath: /data}]
- name: reader
  image: busybox
  command: ["sh", "-c", "sleep 5; cat /data/msg.txt; sleep 3600"]
  volumeMounts: [{name: shared, mountPath: /data}]
```

Volumes — emptyDir and hostPath (cont.)

Step 2 — emptyDir backed by RAM (tmpfs)

```
volumes:  
- name: fast  
  emptyDir:  
    medium: Memory  
    sizeLimit: 64Mi
```

Step 3 — hostPath: access files on the node

```
volumes:  
- name: etc  
  hostPath:  
    path: /etc  
    type: Directory
```

Note Volumes are declared under `spec.volumes` and consumed via `spec.containers[].volumeMounts`. Avoid `hostPath` in multi-tenant clusters — it is a security risk; use it only for DaemonSets and node-level tools.

Volumes — emptyDir and hostPath

✓ Test it

k logs scratch -c reader prints hello-shared, proving the writer and reader containers share the same emptyDir volume.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

WRAP-UP

Day 1 done — you can design and build applications.

Tomorrow: Application Deployment — Deployments, rollouts, Helm & Kustomize.



COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

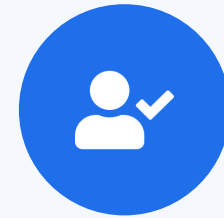
DAY 2

Application Deployment

Deployments · Rollouts · Blue/Green & Canary · Helm · Kustomize · Daemon/StatefulSet (CKAD Domain 2 — 20%)

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.



**Minimum 75%
attendance required**

Recap & Today

- Yesterday (Domain 1): images, Pods, Jobs/CronJobs, multi-container Pods and volumes.
- Today (Domain 2 — 20% of the exam): how applications are deployed, scaled and upgraded.
- From a single Pod to controllers that keep replicas running and roll out new versions safely.
- Packaging with Helm and Kustomize, plus DaemonSets and StatefulSets for specialised workloads.



TOPIC 1

Deployments & ReplicaSets

The controller that keeps your Pods running

Why Deployments?

- A bare Pod is not rescheduled if it dies — you need a controller to keep the desired count.
- A Deployment manages a ReplicaSet, which manages the Pods — a three-tier ownership chain.
- `kubectl scale` changes the replica count; the ReplicaSet controller reconciles immediately.
- Old ReplicaSets are kept for rollback (`revisionHistoryLimit`, default 10).

The Ownership Chain

Deployment

- You declare desired state
- replicas + Pod template
- Owns one active ReplicaSet

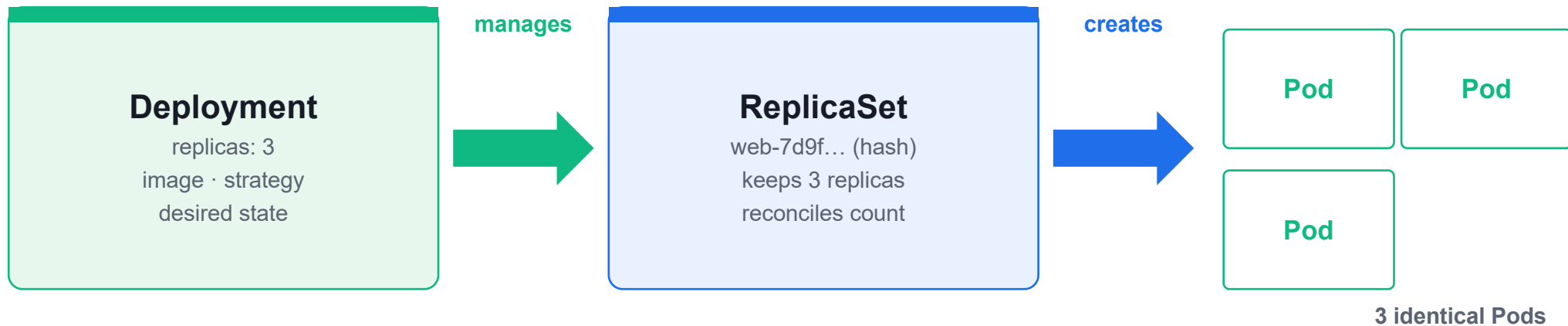
ReplicaSet

- Keeps N Pods running
- Hash-suffixed name
- Self-heals deleted Pods

Pod

- The running workload
- Recreated if it dies
- Never edited directly

Deployment → ReplicaSet → Pods



kubectl scale changes the replica count; **set image / set env** mutate the template and roll out a new ReplicaSet.

Deployments and ReplicaSets

LAB 9

Deployments & ReplicaSets

A Deployment manages a ReplicaSet, which manages Pods. CKAD tests the full ownership chain, scaling, environment-variable injection and reading ReplicaSet history — required to debug failed rollouts.

You'll build

Create a Deployment imperatively, scale it, generate YAML, inject env vars, then read its ReplicaSet history.

Key commands: `k create · k scale · k set · » `kubectl`

Deployments and ReplicaSets

Step 1 — Create a Deployment imperatively

```
k create deployment web --image=nginx:1.25 --replicas=3  
k get deploy,rs,pod -l app=web
```

Step 2 — Scale up and down

```
k scale deployment web --replicas=5  
k scale deployment web --replicas=2  
k get pods -l app=web
```

Step 3 — Generate Deployment YAML and apply

```
k create deployment api --image=httpd:2.4 --replicas=2 $do > api.yaml  
k apply -f api.yaml
```

Deployments and ReplicaSets (cont.)

Step 4 — Inject an env var and observe ReplicaSet history

```
k set env deployment/api APP_COLOR=blue  
k get rs -l app=api      # old RS 0/0/0, new RS 2/2/2
```

Note kubectl set env / set image mutate the Pod template and trigger a new ReplicaSet (a rolling update). Old ReplicaSets are kept for rollback — revisionHistoryLimit (default 10) controls how many.

Deployments and ReplicaSets

✓ Test it

k get deploy,rs,pod -l app=web shows one Deployment owning one ReplicaSet owning the Pods, and k get rs -l app=api shows two ReplicaSets after the env change.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 2

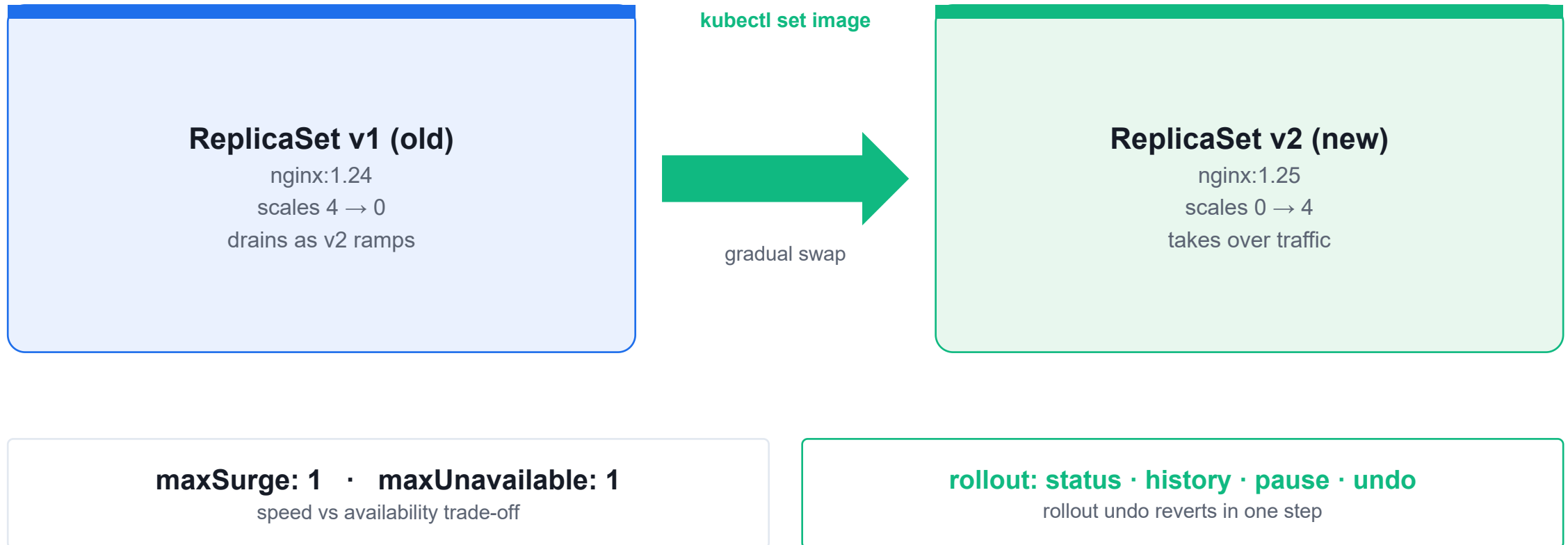
Rolling Updates & Rollback

Zero-downtime upgrades, one ReplicaSet at a time

How a Rolling Update Works

- `kubectl set image` mutates the Pod template — this creates a new ReplicaSet and starts a rollout.
- `maxSurge` = extra Pods allowed above desired; `maxUnavailable` = Pods allowed down — together they tune speed vs availability.
- `kubectl rollout status` watches progress; rollout history lists every revision.
- rollout pause batches several changes into one ReplicaSet; rollout undo reverts instantly.

Rolling Update — One ReplicaSet at a Time



Rolling Updates and Rollback

LAB 10

Rolling Updates & Rollback

A Deployment performs zero-downtime upgrades by gradually replacing Pods one ReplicaSet at a time. CKAD tests maxSurge, maxUnavailable, rollout pause/resume, history inspection and rollback — often as a timed multi-step question.

You'll build

Tune the rolling-update strategy, trigger an image update, inspect history, pause/resume, then roll back.

Key commands: `k create` · `k patch` · `k set` · `k rollout` · » Pausing

Rolling Updates and Rollback

Step 1 — Create the initial Deployment

```
k create deployment web --image=nginx:1.24 --replicas=4
k rollout status deployment/web
```

Step 2 — Tune the rolling-update strategy

```
k patch deployment web -p \
'{"spec":{"strategy":{"rollingUpdate":{"maxSurge":1,"maxUnavailable":1}}}}'
```

Step 3 — Trigger an image update and watch the rollout

```
k set image deployment/web nginx=nginx:1.25
k rollout status deployment/web
k get rs -l app=web          # old RS drains to 0, new RS ramps to 4
```

Rolling Updates and Rollback (cont.)

Step 4 — View history, pause, batch changes, resume

```
k rollout history deployment/web
k rollout pause deployment/web
k set image deployment/web nginx=nginx:1.26
k set env deployment/web APP_ENV=production
k rollout resume deployment/web
```

Step 5 — Roll back

```
k rollout undo deployment/web
k rollout undo deployment/web --to-revision=1
```

Note Pausing batches multiple mutations into a single new ReplicaSet — one rollout, not two. rollout undo reverts to the previous (or a specified) revision instantly.

Rolling Updates and Rollback

✓ Test it

k rollout history deployment/web lists each revision, and k rollout undo returns the Deployment to the prior image with zero downtime.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Lunch Break

1 hour



TOPIC 3

Blue/Green & Canary

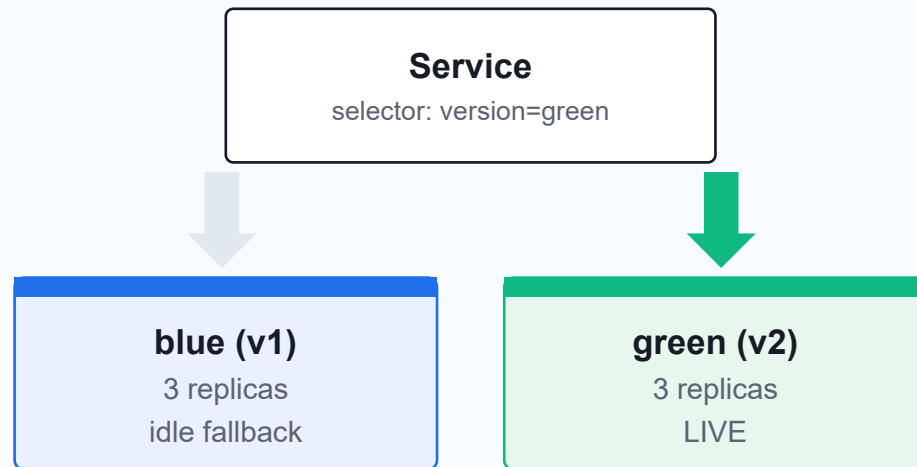
Release strategies built from Deployments + Services

Beyond the Default Rollout

- Blue/Green runs two complete versions; a Service selector switch flips 100% of traffic atomically.
- Canary sends a small slice of traffic to the new version by running it at a low replica ratio.
- Both are built from plain Deployments and Services using labels — no extra tooling for CKAD.
- Rollback is instant: re-patch the selector (blue/green) or scale the canary back down.

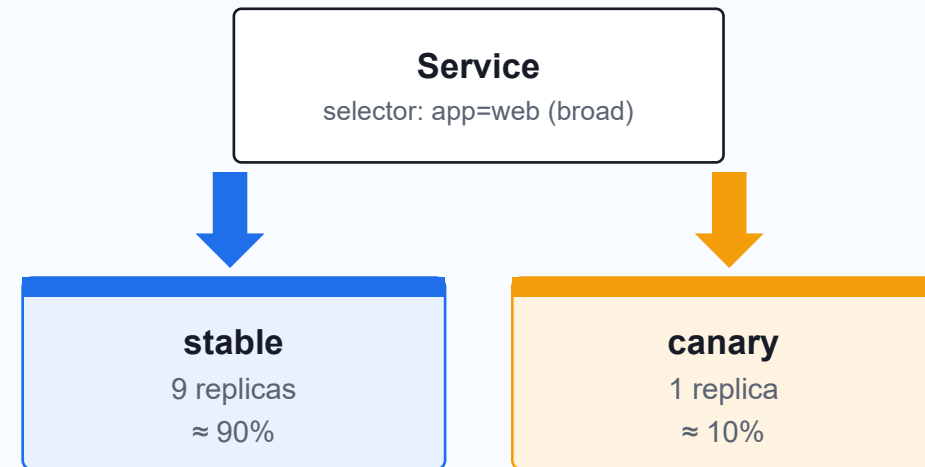
Blue/Green vs Canary

Blue / Green — atomic switch



patch selector → flip 100% (rollback = patch back)

Canary — split by replica ratio



promote = scale canary up, stable down

Blue/Green flips **all traffic** with a selector patch; canary shifts **a fraction** by replica ratio behind one Service.

Blue/Green Deployment

LAB 11

Blue/Green Deployment

Blue/Green keeps two complete copies of the app alive at once. A Service selector switch flips 100% of traffic from blue to green in one atomic operation — with instant rollback if anything breaks.

You'll build

Deploy blue (live) and green (idle), smoke-test green, flip the Service selector, then roll back with one patch.

Key commands: `k apply · GREEN_POD=$(k get · k patch · » Cutover`

Blue/Green Deployment

Step 1 — Deploy blue (current production) behind a Service

```
# Deployment web-blue (labels app=web, version=blue) + Service web
# selector: {app: web, version: blue}
k apply -f blue.yaml
k get pods -l app=web --show-labels
```

Step 2 — Deploy green (next version, no live traffic yet)

```
# Deployment web-green (labels app=web, version=green), nginx:1.25
k apply -f green.yaml
k get pods -l app=web --show-labels
```

Step 3 — Smoke-test green directly before cutover

```
GREEN_POD=$(k get pod -l version=green -o jsonpath='{.items[0].metadata.name}')
k exec $GREEN_POD -- curl -sI localhost:80 | head -2
```

Blue/Green Deployment (cont.)

Step 4 — Flip the Service to green (atomic cutover)

```
k patch service web -p '{"spec":{"selector":{"app":"web","version":"green"}}}'  
k describe svc web | grep Selector
```

Step 5 — Roll back instantly if needed

```
k patch service web -p '{"spec":{"selector":{"app":"web","version":"blue"}}}'
```

Note Cutover and rollback are both a single kubectl patch service selector update — atomic, milliseconds, no Pod churn. Blue stays running as an instant fallback.

Blue/Green Deployment

✓ Test it

After the patch, `k describe svc web | grep Selector` shows `version=green`, and patching it back to `version=blue` restores the old version instantly.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Canary Deployment

LAB 12**Canary Deployment**

A canary release sends a small fraction of live traffic to a new version while the bulk stays on stable. In Kubernetes the split is approximated by replica ratios behind one broad Service selector — no special tooling required.

You'll build

Run a 9-replica stable and 1-replica canary behind one Service, verify the ~10% split, then promote.

Key commands: `k apply · k run · k scale · » Traffic`

Canary Deployment

Step 1 — Stable Deployment (90% of traffic)

```
# Deployment web-stable: replicas 9, labels app=web,track=stable
# Service web: selector {app: web}, port 5678
k apply -f stable.yaml
```

Step 2 — Canary Deployment (10% of traffic)

```
# Deployment web-canary: replicas 1, labels app=web,track=canary
k apply -f canary.yaml
k get pods -l app=web --show-labels
```

Step 3 — Verify the traffic split

```
k run probe --image=busybox --restart=Never -it --rm -- sh -c \
  'for i in $(seq 1 30); do wget -qO- web:5678; done | sort | uniq -c'
# ~27 stable, ~3 canary
```

Canary Deployment (cont.)

Step 4 — Promote: scale up canary, retire stable

```
k scale deployment web-canary --replicas=9  
k scale deployment web-stable --replicas=0
```

Note Traffic split is approximated by replica ratio (9:1 $\approx$ 90/10). Promotion is just `kubectl scale` on both Deployments — no Service change. For precise percentage splits use a service mesh (Istio/Linkerd) — beyond CKAD scope.

Canary Deployment

✓ Test it

The probe loop returns roughly 90% stable and 10% canary, and after scaling, all traffic shifts to the canary version with no Service edit.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 4

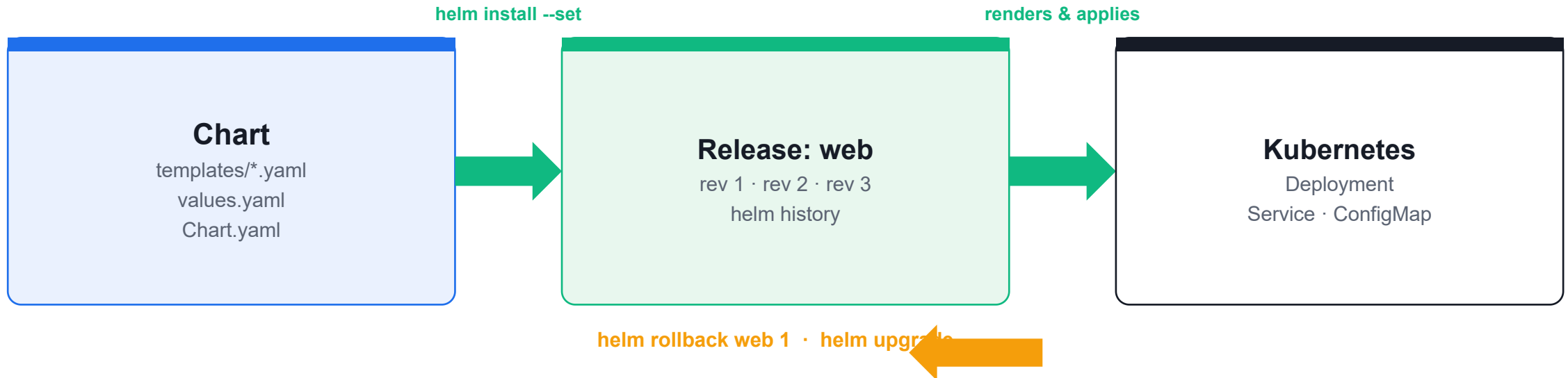
Helm

The Kubernetes package manager

Charts, Releases & Revisions

- A chart is a bundle of templated YAML; a release is one deployed instance of a chart.
- Workflow: repo add → install → upgrade → rollback → uninstall.
- --set overrides values at install/upgrade; --reuse-values keeps prior overrides.
- helm history tracks every revision; helm get manifest shows what was actually applied.

Helm — Chart, Release, Revisions



install → **upgrade** → **rollback** → **uninstall**. `--set` overrides values; `--reuse-values` keeps prior overrides.

Helm — Install, Upgrade, Rollback

LAB 13

Helm

Helm is the Kubernetes package manager. A chart is a bundle of templated YAML; a release is a deployed instance. CKAD tests install, upgrade, rollback, list and value overrides — all under exam time pressure.

You'll build

Add a chart repo, install a release with overrides, inspect it, upgrade, roll back, then uninstall.

Key commands: `helm repo · helm install · helm get · helm upgrade · » --reuse-values`

Helm — Install, Upgrade, Rollback

Step 1 — Add a chart repository

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/nginx | head -5
```

Step 2 — Install a release with value overrides

```
helm install web bitnami/nginx \
  --set service.type=ClusterIP --set replicaCount=2
helm list
```

Step 3 — Inspect the generated manifests and values

```
helm get manifest web | head -40
helm get values web
```

Helm — Install, Upgrade, Rollback (cont.)

Step 4 — Upgrade, then roll back

```
helm upgrade web bitnami/nginx --set replicaCount=4 --reuse-values  
helm history web  
helm rollback web 1  
helm uninstall web
```

Note `--reuse-values` preserves prior overrides while changing only the new one. `helm history <release>` tracks every revision; `rollback` creates a new revision rather than deleting history.

Helm — Install, Upgrade, Rollback

✓ Test it

helm history web lists each revision with its status, and helm rollback web 1 returns the release to its first revision as a new history entry.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 5

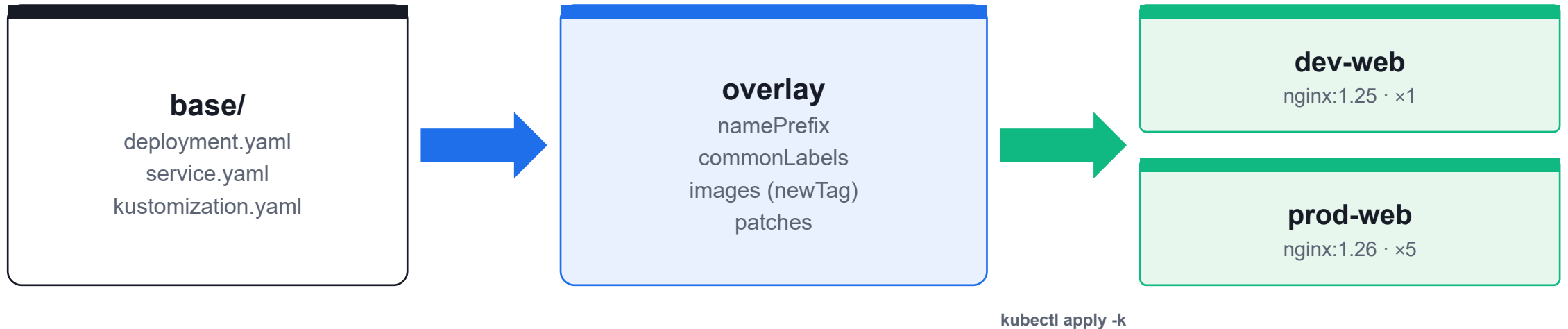
Kustomize

Environment overlays without templating

Base + Overlays

- Kustomize reuses one base set of manifests and layers environment-specific changes on top.
- `namePrefix` and `commonLabels` are applied to every resource in the overlay.
- The `images` block overrides image tags without editing the base files; patches do strategic merges.
- Built into `kubectl`: `kubectl apply -k` and `kubectl kustomize` — no extra binary required.

Kustomize — Base + Overlays



One **base**, many **overlays**. `apply -k` and `kubectl kustomize` are built into `kubectl` — no extra binary.

Kustomize Overlays

LAB 14**Kustomize**

Kustomize reuses a single base set of manifests and applies environment-specific overlays without templating. It is built into kubectl (-k). CKAD tests namePrefix, commonLabels, images and strategic-merge patches.

You'll build

Build a base, add dev and prod overlays (prefix, labels, image tag, replica patch), preview, then apply.

Key commands: resources: · kubectl kustomize · » `namePrefix`

Kustomize Overlays

Step 1 — Base kustomization (shared by all environments)

```
# base/kustomization.yaml
resources:
- deployment.yaml
- service.yaml
```

Step 2 — Dev overlay (name prefix + common label)

```
# overlays/dev/kustomization.yaml
resources:
- ../../base
namePrefix: dev-
commonLabels:
  env: dev
```

Kustomize Overlays (cont.)

Step 3 — Prod overlay (image bump + replica patch)

```
# overlays/prod/kustomization.yaml
resources:
- ../../base
namePrefix: prod-
commonLabels: {env: prod}
images:
- name: nginx
  newTag: "1.26"
patches:
- path: replica-patch.yaml
```

Step 4 — Preview, then apply

```
kubectl kustomize overlays/prod | grep -E "name:|replicas:image:"
kubectl apply -k overlays/dev
kubectl apply -k overlays/prod
```

Kustomize Overlays (cont.)

Note namePrefix and commonLabels apply to every resource in the overlay. The images block overrides tags without editing base files. apply -k / kubectl kustomize are built into kubectl — no extra binary.

Kustomize Overlays

✓ Test it

`kubectl kustomize overlays/prod` shows name `prod-web`, 5 replicas and `nginx:1.26`, while the dev overlay stays at 1 replica and `nginx:1.25`.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 6

DaemonSets & StatefulSets

Node-wide agents & stable-identity workloads

Specialised Controllers

- A DaemonSet runs exactly one Pod per matching node — log collectors, CNI agents, node exporters.
- A StatefulSet gives Pods stable names (db-0, db-1) and ordered, graceful start/stop.
- A StatefulSet needs a headless Service (clusterIP: None) for per-Pod DNS.
- Scale-down always removes the highest ordinal first; scale-up adds the next in order.

Pick the Right Controller

Deployment

- Stateless, interchangeable
- Any number of replicas
- Rolling updates

DaemonSet

- One Pod per node
- Node-level agents
- Scales with the cluster

StatefulSet

- Stable identity + storage
- Ordered start/stop
- Databases, queues

DaemonSet vs StatefulSet

DaemonSet — one Pod per node

node-1

agent Pod

node-2

agent Pod

node-3

agent Pod

StatefulSet — ordered, stable names

headless Service (db)

clusterIP: None

db-0

db-0.db.svc.cluster.local

db-1

db-1.db.svc.cluster.local

db-2

db-2.db.svc.cluster.local

created db-0→db-2 · deleted db-2→db-0

DaemonSet = **one Pod per matching node**; StatefulSet = **stable identity + ordered start/stop**.

DaemonSets and StatefulSets

LAB 15

DaemonSets & StatefulSets

A DaemonSet runs exactly one Pod per matching node (log collectors, CNI agents). A StatefulSet gives Pods stable identities and ordered start/stop (databases). CKAD expects you to know when to use each and write the YAML.

You'll build

Run a DaemonSet on every node, then a StatefulSet with a headless Service, stable DNS and ordered scaling.

Key commands: `k apply · nslookup db-0.db.default.svc.cluster.local · » DaemonSet`

DaemonSets and StatefulSets

Step 1 — DaemonSet on every node

```
# DaemonSet node-agent, image busybox
k apply -f ds.yaml
k get ds,pods -l app=node-agent -o wide # DESIRED = node count
```

Step 2 — Headless Service for the StatefulSet

```
# Service db: clusterIP: None, selector app=db, port 5432
k apply -f headless.yaml
```

Step 3 — StatefulSet with stable identities

```
# StatefulSet db: serviceName db, replicas 3
k apply -f sts.yaml
k rollout status sts/db
k get pods -l app=db # db-0 → db-1 → db-2 in order
```


DaemonSets and StatefulSets (cont.)

Step 4 — Verify stable per-Pod DNS and scale

```
nslookup db-0.db.default.svc.cluster.local
k scale sts db --replicas=4      # adds db-3
k scale sts db --replicas=2      # removes db-3 then db-2
```

Note DaemonSet = one Pod per matching node. StatefulSet = stable names (db-0, db-1) and ordered start/stop; a headless Service (clusterIP: None) gives each Pod its own DNS. Scale-down always removes the highest ordinal first.

DaemonSets and StatefulSets

✓ Test it

k get pods -l app=db shows db-0, db-1, db-2 created in order, and nslookup db-0.db.default.svc.cluster.local resolves to a single Pod IP.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

WRAP-UP

Day 2 done — you can deploy, scale and upgrade applications.

Tomorrow: Observability and Configuration & Security — probes, logging, ConfigMaps, Secrets, RBAC.



COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

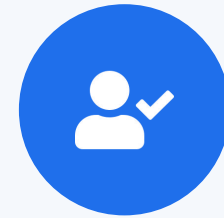
DAY 3

Observability + Config & Security

Probes · Logging · Metrics · Debug · ConfigMaps · Secrets · SecurityContext · RBAC (CKAD Domains 3 & 4 — 40%)

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.



**Minimum 75%
attendance required**

Recap & Today

- Yesterday (Domain 2): Deployments, rollouts, release strategies, Helm, Kustomize, Daemon/StatefulSet.
- Morning — Domain 3 Observability (15%): probes, logging, metrics, debugging, API deprecations.
- Afternoon — Domain 4 Configuration & Security (25%): ConfigMaps, Secrets, securityContext, RBAC, quotas.
- Together these two domains are 40% of the CKAD exam — the largest single day.



DOMAIN 3

Observability & Maintenance

Probes · logging · metrics · debugging · API versions (15%)



TOPIC 1

Health Probes

liveness · readiness · startup

Three Probes, Three Jobs

- livenessProbe — restarts the container when it has hung or deadlocked.
- readinessProbe — removes the Pod from Service endpoints until it can serve traffic (no restart).
- startupProbe — gives slow apps time to boot; it blocks the other two until it succeeds.
- Each probe uses one handler: httpGet, tcpSocket or exec — pick by what the container exposes.

Three Probes, Three Jobs

startupProbe

runs first, exclusively



Holds off liveness/readiness; kills the container if the app never finishes starting.

livenessProbe

is the container alive?



On failure → restart the container (fixes hangs / deadlocks).

readinessProbe

ready to serve traffic?



On failure → remove the Pod from Service endpoints (no restart).

Handlers: httpGet · tcpSocket · exec **Timing:** initialDelaySeconds · periodSeconds · failureThreshold

Liveness, Readiness & Startup Probes

LAB 16

Probes

Kubernetes uses three probes to manage health: livenessProbe restarts a failed container, readinessProbe removes it from Service endpoints, startupProbe gives slow apps time to boot. CKAD tests all three handlers (httpGet, tcpSocket, exec) and timing.

You'll build

Add HTTP readiness+liveness probes, trigger a liveness restart, add a TCP probe, then a startupProbe with exec.

Key commands: readinessProbe: · k exec · startupProbe: · » Three

Liveness, Readiness & Startup Probes

Step 1 — HTTP liveness + readiness probe

```
readinessProbe:
  httpGet: {path: /, port: 80}
  initialDelaySeconds: 2
  periodSeconds: 5
livenessProbe:
  httpGet: {path: /, port: 80}
  initialDelaySeconds: 10
  periodSeconds: 10
  failureThreshold: 3
```

Step 2 — Trigger a liveness failure (watch RESTARTS climb)

```
k exec web-probes -- rm /usr/share/nginx/html/index.html
sleep 35
k get pod web-probes
```

Liveness, Readiness & Startup Probes (cont.)

Step 3 — TCP socket probe

```
readinessProbe:
  tcpSocket: {port: 6379}
  periodSeconds: 5
```

Step 4 — startupProbe for slow-starting apps

```
startupProbe:
  exec: {command: ["cat", "/tmp/ready"]}
  failureThreshold: 30
  periodSeconds: 5
```

Note Three handlers — httpGet, tcpSocket, exec. readinessProbe failure removes the Pod from Service endpoints without restarting it; livenessProbe failure restarts the container; startupProbe blocks the other two until the app is initialised.

Liveness, Readiness & Startup Probes

✓ Test it

Deleting the index page makes the liveness probe fail three times and the container restarts (RESTARTS increments), while the TCP and startup Pods reach READY 1/1.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 2

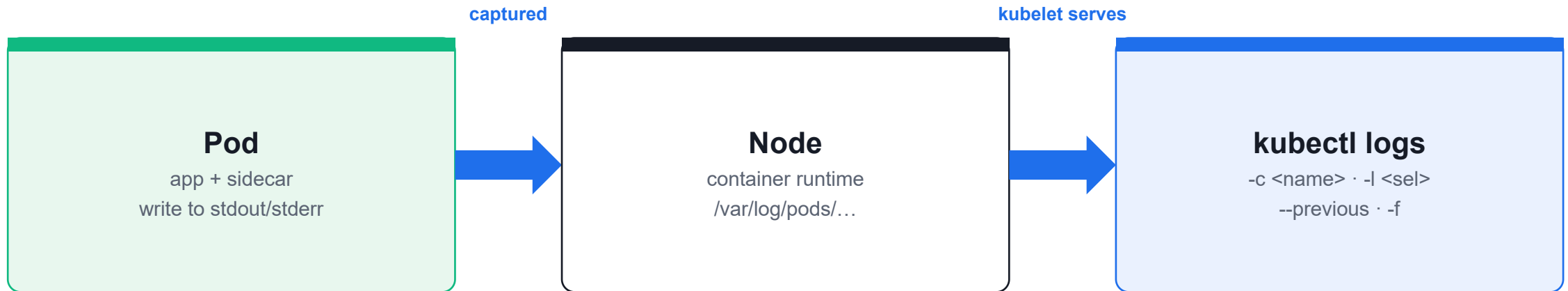
Logging

kubectl logs — your first debugging tool

Reading Container Logs

- Logs are whatever the container writes to stdout/stderr; the kubelet captures and serves them.
- `--tail=N` for recent lines; `-f` to stream live (like `tail -f`).
- `--previous` reads the prior terminated container — the key to diagnosing `CrashLoopBackOff`.
- `-c <container>` selects one container; `-l <selector>` aggregates logs across matching Pods.

Where Logs Come From



Logs are whatever the container prints to **stdout/stderr**. Use **--previous** for a crashed container.

Container Logging

LAB 17**Logging**

kubectl logs is the first debugging tool on the exam. Read logs from single and multi-container Pods, follow a live stream, retrieve logs from a crashed container, and aggregate logs across a Deployment.

You'll build

Read and tail logs, stream with -f, recover crash logs with --previous, then aggregate by container and label.

Key commands: `k logs · » `--tail=N``

Container Logging

Step 1 — Read recent lines and follow a live stream

```
k logs noisy --tail=10
k logs -f noisy &          # -f streams like tail -f
kill %1
```

Step 2 — Retrieve logs from a crashed container

```
k logs crasher --previous | head -5
```

Step 3 — Multi-container Pod logs

```
k logs multi -c app | head -3
k logs multi -c sidecar | head -3
k logs multi --all-containers=true --prefix=true | head -8
```

Container Logging (cont.)

Step 4 — Aggregate logs across a Deployment

```
k logs deployment/fleet --tail=3
k logs -l app=fleet --prefix=true --tail=2
```

Note --tail=N for recent lines, -f to stream, --previous for crash loops, -c <name> for a specific container, -l <selector> to aggregate matching Pods. --prefix=true labels each line with [pod/container].

Container Logging

✓ Test it

`k logs crasher --previous` shows the message printed before the crash, and `k logs -l app=fleet --prefix=true` interleaves lines from all three Pods.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 3

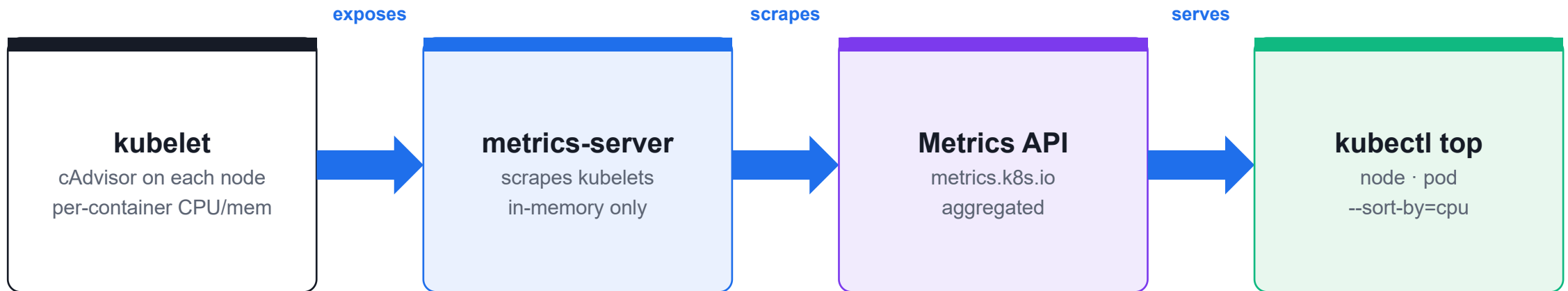
Metrics

kubectl top & the metrics-server add-on

Resource Usage at a Glance

- `kubectl top` shows live CPU and memory for nodes and Pods — it requires metrics-server.
- metrics-server scrapes each kubelet's cAdvisor and serves the aggregated Metrics API.
- Sort with `--sort-by=cpu` / `--sort-by=memory`; span all namespaces with `-A`.
- On Killercode's self-signed certs, patch the Deployment with `--kubelet-insecure-tls`.

kubectl top Needs metrics-server



No metrics-server = **kubectl top fails**. On Killercode add **--kubelet-insecure-tls**.

kubectl top and Metrics Server

LAB 18**Metrics**

kubectl top shows real-time CPU and memory for nodes and Pods. It requires the metrics-server add-on. CKAD tests installing metrics-server, reading node/Pod metrics, and sorting by resource usage.

You'll build

Install metrics-server, read node metrics, generate CPU load, then sort Pod metrics by CPU and memory.

Key commands: `kubectl apply · k top · k run · » `kubectl`

kubectl top and Metrics Server

Step 1 — Install metrics-server (and trust kubelet certs on Killercode)

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl patch -n kube-system deployment metrics-server --type=json -p='
[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'
until kubectl top node 2>/dev/null; do sleep 5; done
```

Step 2 — Top nodes

```
k top node          # CPU cores/% and memory bytes/% per node
```

Step 3 — Generate load, then top Pods sorted

```
k run cpu-burner --image=busybox -- sh -c 'while true; do :; done'
sleep 30
k top pod --sort-by=cpu
k top pod --sort-by=memory
k top pod -A --sort-by=cpu | head -10
```

kubectl top and Metrics Server (cont.)

Note kubectl top needs metrics-server running. --kubelet-insecure-tls is required on Killercode's self-signed certs. cpu-burner should top the CPU sort; -A spans all namespaces.

kubectl top and Metrics Server

✓ Test it

After ~30s, k top node reports CPU/memory and k top pod --sort-by=cpu lists cpu-burner at the top of the usage ranking.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 4

Debugging

Diagnose and fix broken workloads under time pressure

The Three Failures You Will See

- ImagePullBackOff — wrong image name/tag; confirm in describe → Events, then fix and recreate.
- CrashLoopBackOff — container exits non-zero; read `kubectl logs --previous`.
- OOMKilled — exceeded the memory limit; raise `resources.limits.memory`.
- `kubectl get events --sort-by=.lastTimestamp` and `kubectl debug` (ephemeral container) are your fast tools.

Diagnose: Three Classic Failures

ImagePullBackOff

wrong image name / tag



describe pod → 'Failed to pull' → fix the image and recreate.

CrashLoopBackOff

container exits non-zero



kubectl logs --previous → fix the command / config.

OOMKilled

exceeded memory limit



describe → Reason: OOMKilled → raise resources.limits.memory.

Fast triage: `kubectl get events --sort-by=.lastTimestamp` · `kubectl debug` (ephemeral container for distroless)

Debugging Pods and Events

LAB 19

Debugging

CKAD regularly includes broken workloads you must diagnose and fix under time pressure. Identify and resolve ImagePullBackOff, CrashLoopBackOff and OOMKilled, and use `kubectl debug` for live triage.

You'll build

Diagnose three classic failures with `describe/logs/events`, then inject an ephemeral debug container.

Key commands: `k describe · k get · » ImagePullBackOff`

Debugging Pods and Events

Step 1 — ImagePullBackOff (wrong image name)

```
k describe pod typo | tail -15      # 'Failed to pull image'  
k delete pod typo --force --grace-period=0  
k run typo --image=nginx:1.25      # corrected
```

Step 2 — CrashLoopBackOff (container exits non-zero)

```
k describe pod crash | grep -A3 "Last State"  
k logs crash --previous
```

Step 3 — OOMKilled (exceeds memory limit)

```
k describe pod hungry | grep -A4 "Last State"  # Reason: OOMKilled  
# fix: raise resources.limits.memory
```

Debugging Pods and Events (cont.)

Step 4 — Cluster-wide events + ephemeral debug container

```
k get events -A --sort-by=.lastTimestamp | tail -20  
k get events --field-selector type=Warning  
k debug crash -it --image=busybox --target=crash -- sh
```

Note ImagePullBackOff → wrong image/tag. CrashLoopBackOff → use logs --previous. OOMKilled → raise the memory limit. kubectl debug injects an ephemeral container sharing the target's namespaces — vital for distroless images with no shell.

Debugging Pods and Events

✓ Test it

k describe reveals the correct failure reason for each Pod (ImagePullBackOff, CrashLoopBackOff, OOMKilled), and k debug opens a shell inside the running Pod.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 5

API Deprecations

Find the right apiVersion with kubectl

APIs Change — Know How to Look Them Up

- Old API versions are deprecated and eventually removed (e.g. extensions/v1beta1 Ingress, gone in 1.22).
- `kubectl api-resources` shows the correct `apiVersion` for every resource in its `APIVERSION` column.
- `kubectl api-versions` lists every version the cluster serves.
- `kubectl explain <resource>.<field> --recursive` documents fields without leaving the terminal.

Find the Right apiVersion

Pod · Service · ConfigMap · Secret	→	v1
Deployment · StatefulSet · DaemonSet	→	apps/v1
Job · CronJob	→	batch/v1
Ingress · NetworkPolicy	→	networking.k8s.io/v1
Role · RoleBinding · ClusterRole	→	rbac.authorization.k8s.io/v1

Read the **APIVERSION** column of `kubectl api-resources`; explore fields with `kubectl explain`.

API Deprecations & kubectl explain

LAB 20

API Deprecations

The Kubernetes API evolves — old versions are deprecated and removed. CKAD tests kubectl explain, api-resources and api-versions. You must find the correct apiVersion for any resource on exam day using only kubectl.

You'll build

List resources and versions, explore schemas with explain, detect a removed API, and look up the current version.

Key commands: `k api-resources · k explain · Pod/Service/ConfigMap/Secret/SA/Quota/LimitRange -> · » `kubectl`

API Deprecations & kubectl explain

Step 1 — List resources, groups and served versions

```
k api-resources --api-group=apps
k api-resources --api-group=batch
k api-versions | sort
```

Step 2 — Explore a schema with explain

```
k explain deployment.spec.strategy.rollingUpdate
k explain pod.spec.containers.resources --recursive | head -30
```

Step 3 — Detect a removed API and find the replacement

```
# extensions/v1beta1 Ingress was removed in 1.22
k explain ingress --api-version=networking.k8s.io/v1 | head -10
k api-resources | grep -i ingress
```

API Deprecations & kubectl explain (cont.)

Step 4 — Stable apiVersions to memorise (v1.35)

```
Pod/Service/ConfigMap/Secret/SA/Quota/LimitRange -> v1
Deployment/StatefulSet/DaemonSet/ReplicaSet      -> apps/v1
Job/CronJob                                       -> batch/v1
Ingress/NetworkPolicy                           -> networking.k8s.io/v1
Role/RoleBinding/ClusterRole(+Binding)          -> rbac.authorization.k8s.io/v1
HorizontalPodAutoscaler                         -> autoscaling/v2
```

Note kubectl api-resources shows the correct apiVersion in its APIVERSION column — the exam-legal way to look it up. kubectl explain <res>.<field> documents fields without leaving the terminal.

API Deprecations & kubectl explain

✓ Test it

`k api-resources | grep -i ingress` shows `networking.k8s.io/v1`, and `k explain deployment.spec.strategy.rollingUpdate` prints the `maxSurge/maxUnavailable` fields.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Lunch Break

1 hour



DOMAIN 4

Configuration & Security

ConfigMaps · Secrets · securityContext · ServiceAccounts · RBAC · quotas (25%)



TOPIC 6

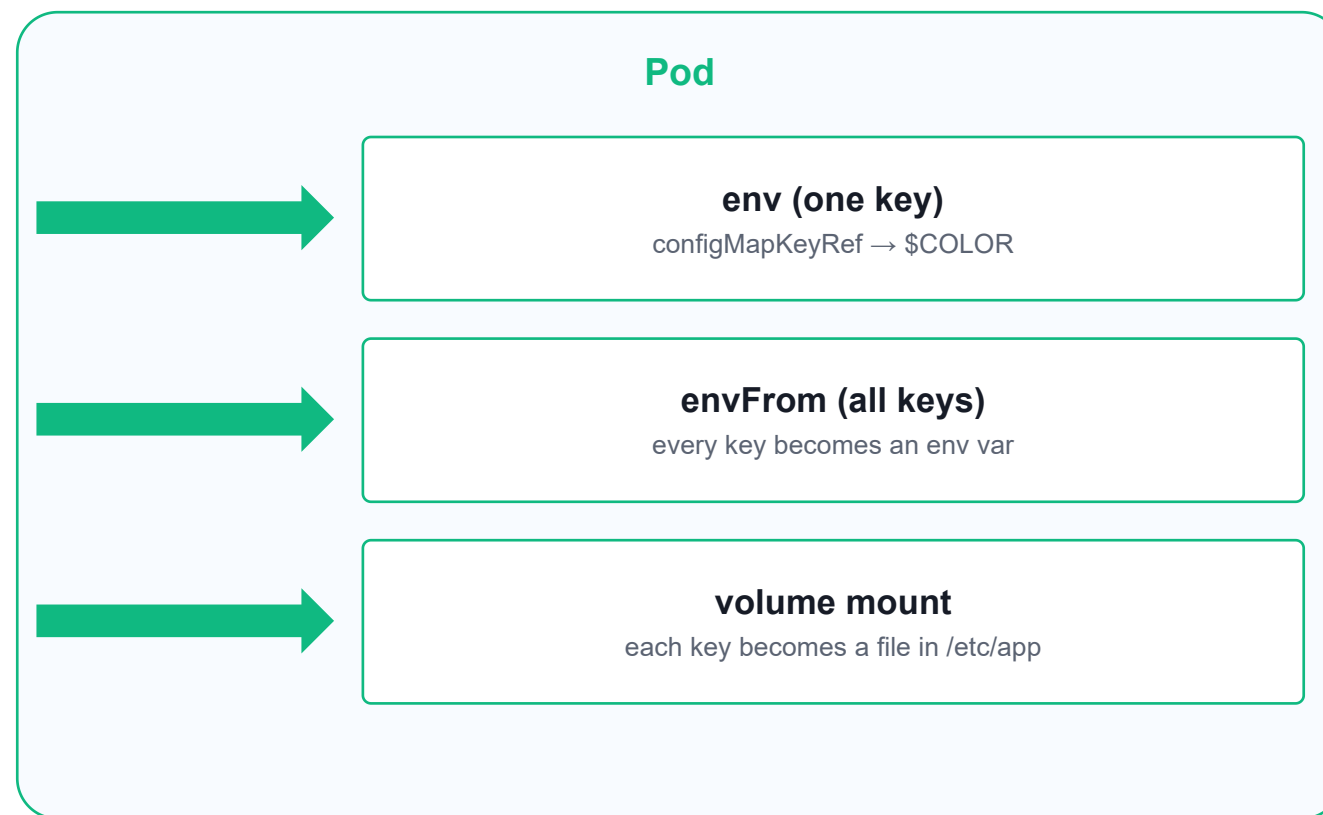
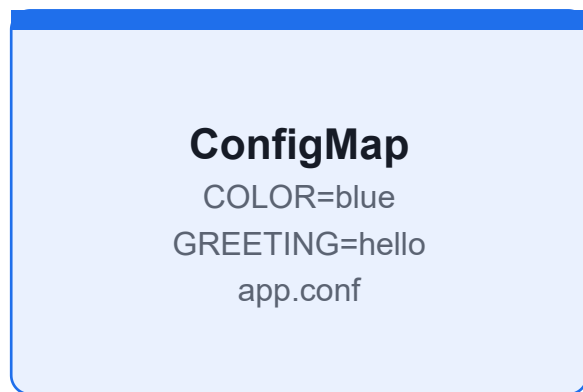
ConfigMaps

Inject non-secret configuration into Pods

Decouple Config from Image

- A ConfigMap holds non-secret key/value data; create it with `--from-literal`, `--from-file` or `--from-env-file`.
- Inject one key with `configMapKeyRef`, all keys with `envFrom`, or mount it as a file volume.
- Volume-mounted ConfigMaps update live (~60s); env-var injections are fixed at Pod startup.
- Keep config out of the image so the same image runs in every environment.

Three Ways to Consume a ConfigMap



Volume-mounted ConfigMaps **update live (~60s)**; env-var injections are **fixed at Pod startup**.

ConfigMaps — Env & Volume Injection

LAB 21**ConfigMaps**

ConfigMaps inject non-secret configuration into Pods. CKAD tests all three injection styles: a single env var (configMapKeyRef), bulk env vars (envFrom), and file-based volume mounts — and that only file mounts update live.

You'll build

Create ConfigMaps three ways, inject one key, inject all keys with envFrom, mount as files, then update live.

Key commands: `k create · env: · envFrom: · volumeMounts: · » Creation:`

ConfigMaps — Env & Volume Injection

Step 1 — Create ConfigMaps three ways

```
k create configmap app-cfg --from-literal=COLOR=blue --from-literal=GREETING=hello
k create configmap app-conf --from-file=app.conf
k create configmap app-env --from-env-file=env.list
```

Step 2 — Inject a single key as an env var

```
env:
- name: COLOR
  valueFrom:
    configMapKeyRef: {name: app-cfg, key: COLOR}
```

Step 3 — Inject all keys with envFrom

```
envFrom:
- configMapRef: {name: app-cfg}
- configMapRef: {name: app-env}
```

ConfigMaps — Env & Volume Injection (cont.)

Step 4 — Mount a ConfigMap as a file volume

```
volumeMounts:  
- {name: cfg, mountPath: /etc/app}  
volumes:  
- name: cfg  
  configMap: {name: app-conf}
```

Note Creation: --from-literal, --from-file, --from-env-file. Consumption: configMapKeyRef (one key), envFrom (all keys), volume mount (file per key). Volume-mounted ConfigMaps update live (~60s); env-var injections are fixed at Pod startup and need a restart.

ConfigMaps — Env & Volume Injection

✓ Test it

The single-key Pod logs `COLOR=blue`, the `envFrom` Pod shows all four variables, and the mounted ConfigMap appears as files under `/etc/app`.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 7

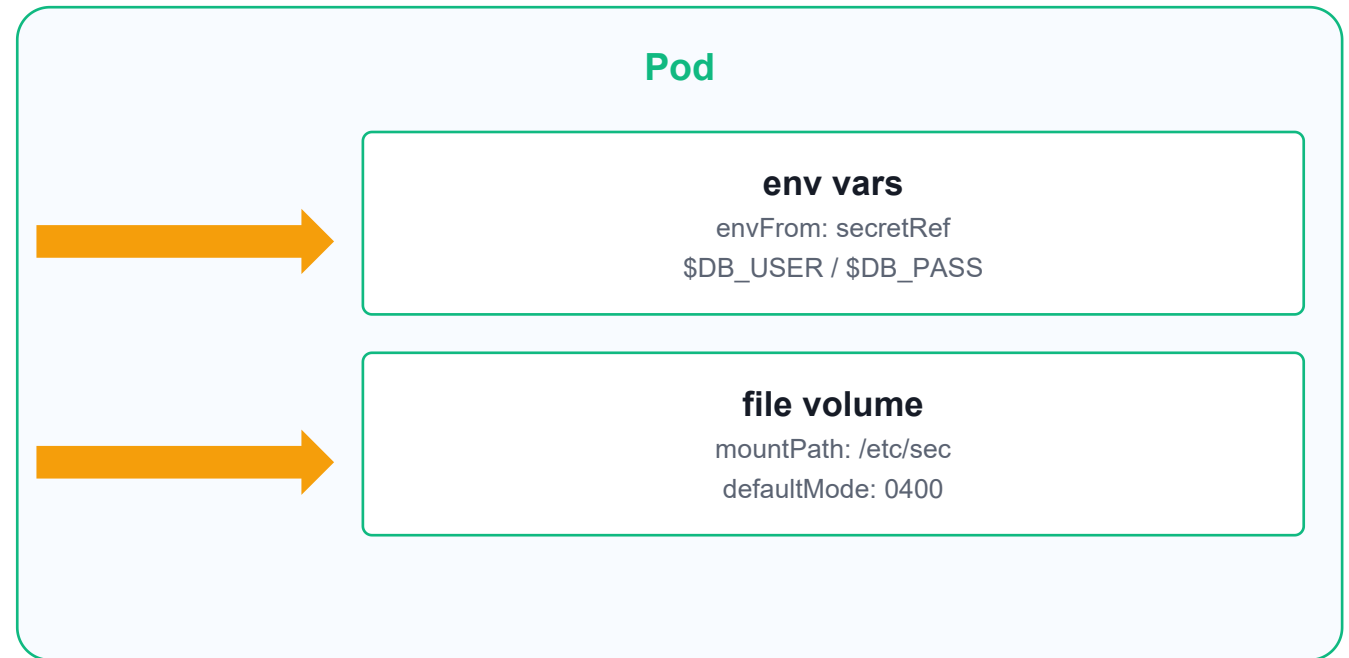
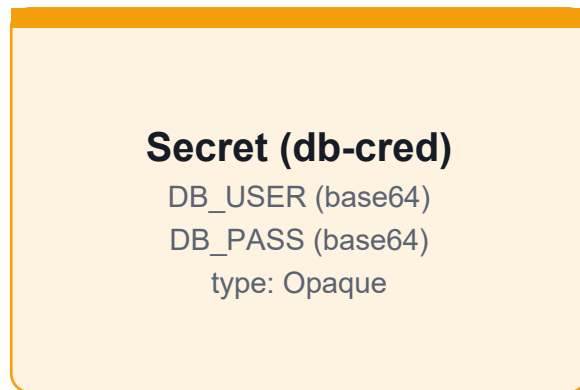
Secrets

Sensitive configuration, guarded by RBAC

Secrets vs ConfigMaps

- Secrets work like ConfigMaps but values are base64-encoded and access is more tightly controlled.
- Types: generic, tls (kubernetes.io/tls) and docker-registry (for imagePullSecrets).
- Inject with envFrom: secretRef (all keys) or secretKeyRef (one key); mount with defaultMode: 0400.
- Base64 is not encryption — restrict access with RBAC and enable encryption at rest.

Secrets — Inject as Env or Files



Secrets are **base64-encoded, not encrypted** — restrict with RBAC. **defaultMode: 0400** = owner read-only.

Secrets

LAB 22**Secrets**

Secrets are like ConfigMaps but base64-encoded with stricter access. CKAD tests generic, tls and docker-registry types, env-var injection, file mounts with defaultMode, and decoding values. Secrets are not encrypted — protect them with RBAC.

You'll build

Create a generic Secret, decode it, inject as env vars, mount with 0400 mode, then create tls and docker-registry Secrets.

Key commands: `k create · envFrom: · volumes: · »` Types:

Secrets

Step 1 — Create a generic Secret and decode a value

```
k create secret generic db-cred \
  --from-literal=DB_USER=admin --from-literal=DB_PASS=s3cr3t
k get secret db-cred -o jsonpath='{.data.DB_PASS}' | base64 -d; echo
```

Step 2 — Inject Secret keys as env vars

```
envFrom:
- secretRef: {name: db-cred}
```

Step 3 — Mount a Secret as files with restrictive mode

```
volumes:
- name: s
  secret:
    secretName: db-cred
    defaultMode: 0400
```

Secrets (cont.)

Step 4 — TLS and docker-registry Secrets

```
k create secret tls demo-tls --cert=tls.crt --key=tls.key
k create secret docker-registry myreg \
  --docker-server=registry.example.com --docker-username=u \
  --docker-password=p --docker-email=u@example.com
```

Note Types: generic, tls (kubernetes.io/tls), docker-registry. envFrom: secretRef injects all keys; secretKeyRef injects one. Reference a registry Secret via spec.imagePullSecrets. Base64 ≠ encryption — restrict with RBAC.

Secrets

✓ Test it

base64 -d on .data.DB_PASS returns s3cr3t, the mounted Secret files are mode 0400, and the tls Secret reports type kubernetes.io/tls.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 8

SecurityContext

Control container identity & privileges

Run Containers Safely

- securityContext sets the user, group and privileges a container runs with.
- Pod-level applies to all containers; container-level overrides one container.
- runAsNonRoot: true blocks any image whose default user is root.
- Hardened pattern: readOnlyRootFilesystem + capabilities.drop:[ALL] + allowPrivilegeEscalation:false.

securityContext — Pod vs Container

Pod

Pod-level securityContext — applies to ALL containers

runAsUser · runAsGroup · fsGroup · runAsNonRoot: true

container A

inherits Pod context

container B (override)

readOnlyRootFilesystem: true
capabilities.drop: [ALL]
allowPrivilegeEscalation: false

Pod-level sets defaults for all containers; **container-level overrides one**. Hardened = nonRoot + read-only FS + drop ALL caps.

SecurityContext

LAB 23**SecurityContext**

securityContext controls the identity and privileges of containers. CKAD asks you to enforce non-root execution, a read-only root filesystem and dropped Linux capabilities — at Pod level (all containers) or container level (one override).

You'll build

Run as a set user/group, enforce runAsNonRoot, lock the root filesystem, then drop all capabilities.

Key commands: securityContext: · » Pod-level

SecurityContext

Step 1 — Run as a specific user and group (Pod level)

```
securityContext:  
  runAsUser: 1000  
  runAsGroup: 3000  
  fsGroup: 2000    # applies to volume mounts
```

Step 2 — Enforce runAsNonRoot (blocks root images)

```
securityContext:  
  runAsNonRoot: true  
# nginx runs as root -> Pod refuses to start
```

Step 3 — Read-only root filesystem (container level)

```
securityContext:  
  readOnlyRootFilesystem: true  
# mount an emptyDir for any writable path (e.g. /tmp)
```

SecurityContext (cont.)

Step 4 — Drop Linux capabilities (hardened container)

```
securityContext:  
  capabilities:  
    drop: ["ALL"]  
    add: ["NET_BIND_SERVICE"]  
  allowPrivilegeEscalation: false
```

Note Pod-level securityContext applies to all containers; container-level overrides one. The CKAD hardened pattern = runAsNonRoot: true + readOnlyRootFilesystem: true + capabilities.drop: [ALL] + allowPrivilegeEscalation: false.

SecurityContext

✓ Test it

The non-root Pod logs uid=1000 gid=3000, the runAsNonRoot nginx Pod refuses to start, and writes to the read-only root filesystem are rejected.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 9

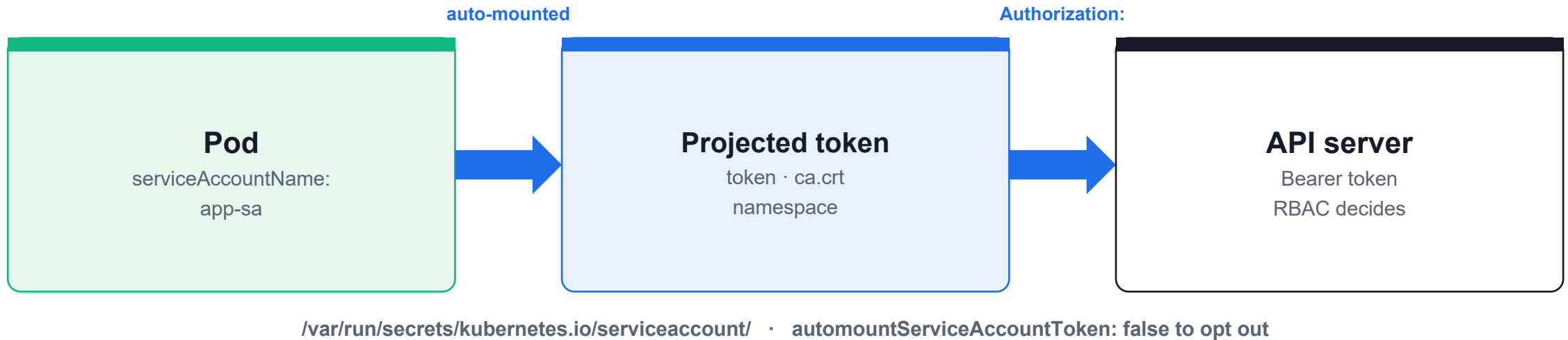
ServiceAccounts

The identity a Pod presents to the API

Pod Identity

- Every Pod runs as a ServiceAccount — the default one has minimal permissions.
- `spec.serviceAccountName` attaches a dedicated ServiceAccount to a Pod.
- A token, CA cert and namespace are projected into `/var/run/secrets/kubernetes.io/serviceaccount/`.
- `automountServiceAccountToken: false` enforces least privilege; `kubectl create token` mints short-lived tokens.

ServiceAccount & Projected Token



Every Pod runs as a ServiceAccount (**default** if unset). No RBAC → the API returns **403 Forbidden**.

ServiceAccounts

LAB 24**ServiceAccounts**

Every Pod runs as a ServiceAccount; the default one has minimal RBAC. CKAD tests creating dedicated ServiceAccounts, attaching them to Pods, disabling the auto-mounted token, and requesting short-lived tokens with `kubectl create token`.

You'll build

Create a ServiceAccount, attach it to a Pod, inspect the projected token, disable auto-mount, then mint a token.

Key commands: `k create · k exec · k patch · TOKEN=$(k create · » `spec.serviceAccountName``

ServiceAccounts

Step 1 — Create a ServiceAccount and attach it to a Pod

```
k create serviceaccount app-sa
# Pod spec:
spec:
  serviceAccountName: app-sa
k get pod app -o jsonpath='{.spec.serviceAccountName}'; echo
```

Step 2 — Inspect the auto-mounted token inside the Pod

```
k exec app -- ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Step 3 — Disable token auto-mount (least privilege)

```
k patch sa app-sa -p '{"automountServiceAccountToken": false}'
```

ServiceAccounts (cont.)

Step 4 — Request a short-lived ad-hoc token

```
TOKEN=$(k create token app-sa --duration=1h)
echo $TOKEN | cut -c1-60
```

Note spec.serviceAccountName attaches a custom SA. Token, CA and namespace are projected under /var/run/secrets/kubernetes.io/serviceaccount/. automountServiceAccountToken: false enforces least privilege; kubectl create token replaces the old Secret-backed tokens (removed in 1.24+).

ServiceAccounts

✓ Test it

`k get pod app -o jsonpath='{.spec.serviceAccountName}'` returns `app-sa`, and `kubectl create token app-sa` prints a short-lived JWT.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 10

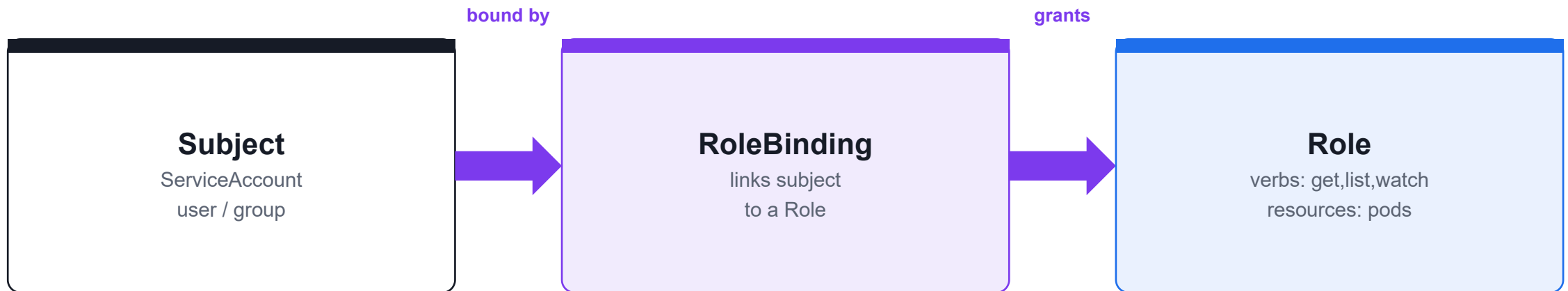
RBAC

Who can do what, on which resources, where

Role-Based Access Control

- A Role/ClusterRole lists allowed verbs (get, list, create, delete...) on resources.
- A RoleBinding/ClusterRoleBinding ties that Role to a subject (ServiceAccount, user or group).
- Role + RoleBinding = one namespace; ClusterRole + ClusterRoleBinding = cluster-wide.
- ClusterRole + RoleBinding limits a cluster role's verbs to a single namespace.
- Validate without logging in: `kubectl auth can-i <verb> <resource> --as=<identity>`.

RBAC — Subject → Binding → Role



Role + RoleBinding = one namespace

ClusterRole + ClusterRoleBinding = cluster-wide

RBAC — Roles & RoleBindings

LAB 25

RBAC

RBAC controls who can do what on which resources. CKAD tests creating Roles, ClusterRoles, RoleBindings and ClusterRoleBindings imperatively, and validating with `kubectl auth can-i --as`. Know namespace- vs cluster-scope.

You'll build

Create a Role + RoleBinding for a ServiceAccount, validate with can-i, then a ClusterRole bound two ways.

Key commands: `k create · k auth · » Role`

RBAC — Roles & RoleBindings

Step 1 — Role: namespace-scoped permissions

```
k create role pod-reader \  
  --verb=get,list,watch --resource=pods -n dev
```

Step 2 — RoleBinding: link Role to a ServiceAccount

```
k create rolebinding viewer-binding \  
  --role=pod-reader --serviceaccount=dev:viewer -n dev
```

Step 3 — Validate with kubectl auth can-i

```
k auth can-i list pods    -n dev      --as=system:serviceaccount:dev:viewer # yes  
k auth can-i delete pods  -n dev      --as=system:serviceaccount:dev:viewer # no  
k auth can-i list pods    -n default  --as=system:serviceaccount:dev:viewer # no
```


RBAC — Roles & RoleBindings (cont.)

Step 4 — ClusterRole bound cluster-wide vs to one namespace

```
k create clusterrole node-reader --verb=get,list --resource=nodes
k create clusterrolebinding nodes-binding \
  --clusterrole=node-reader --serviceaccount=dev:nodes-sa # all namespaces
k create rolebinding dev-node-reader \
  --clusterrole=node-reader --serviceaccount=dev:viewer -n dev # only dev
```

Note Role + RoleBinding = namespace-scoped. ClusterRole + ClusterRoleBinding = cluster-wide. ClusterRole + RoleBinding = the cluster role's verbs limited to one namespace. `kubectl auth can-i <verb> <res> --as=<identity>` validates without logging in.

RBAC — Roles & RoleBindings

✓ Test it

k auth can-i list pods -n dev --as=system:serviceaccount:dev:viewer returns yes while delete pods and cross-namespace list return no.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 11

Quotas & Limits

Cap namespace usage & per-container resources

Govern Resource Consumption

- ResourceQuota caps the aggregate resources a namespace can consume (pods, requests, limits).
- LimitRange enforces per-container defaults, defaultRequests and maximums.
- Without a LimitRange, Pods with no resource block are rejected in a quota-enforced namespace.
- `kubectl describe quota` shows Used vs Hard — the primary troubleshooting view.

ResourceQuota vs LimitRange

Namespace: team-a

ResourceQuota

aggregate ceiling
pods: 5
requests/limits cpu+mem

LimitRange

per-container
default / defaultRequest
max ceiling

Pod ✓

gets defaults injected

Pod ✓

within limits

Pod ✗

6th pod / over max → rejected

ResourceQuota caps **total namespace usage**; LimitRange enforces **per-container defaults & maxima**.

ResourceQuota & LimitRange

LAB 26

Quotas & Limits

ResourceQuota caps the total resources used across a namespace; LimitRange enforces per-container defaults and maximums. CKAD tests both — write the YAML, apply them, and understand the rejection error messages.

You'll build

Apply a ResourceQuota and a LimitRange, watch defaults get injected, then violate the max and the quota.

Key commands: `spec: · k run · for i · » ResourceQuota`

ResourceQuota & LimitRange

Step 1 — ResourceQuota: namespace-wide ceilings

```
spec:
  hard:
    pods: "5"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
```

Step 2 — LimitRange: per-container defaults and maximums

```
spec:
  limits:
    - type: Container
      default: {cpu: 200m, memory: 256Mi}
      defaultRequest: {cpu: 100m, memory: 128Mi}
      max: {cpu: 500m, memory: 512Mi}
```


ResourceQuota & LimitRange (cont.)

Step 3 — Defaults injected; max enforced

```
k run a --image=nginx:1.25 -n team-a
k get pod a -n team-a -o jsonpath='{.spec.containers[0].resources}'; echo
# a container requesting cpu:1 is rejected: 'maximum cpu per Container is 500m'
```

Step 4 — Exceed the quota

```
for i in 1 2 3 4; do k run quota-$i --image=nginx:1.25 -n team-a; done
k run quota-5 --image=nginx:1.25 -n team-a    # 'exceeded quota: team-a-quota'
k describe quota team-a-quota -n team-a      # Used vs Hard
```

Note ResourceQuota limits aggregate usage; exceeding it rejects new Pods. LimitRange enforces per-container min/max/defaults. Without a LimitRange, Pods with no resource block cannot be created in a quota-enforced namespace.

ResourceQuota & LimitRange

✓ Test it

A Pod with no resource block gets requests/limits injected by the LimitRange, the 6th Pod is rejected with exceeded quota, and describe quota shows Used vs Hard.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

WRAP-UP

Day 3 done — you can observe, configure and secure applications.

Tomorrow (half day): Services & Networking, then the final assessment.



COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

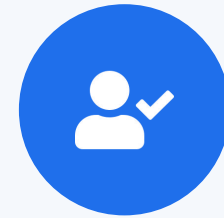
DAY 4 (½ day)

Services & Networking

Services · Service DNS · Ingress with TLS · NetworkPolicy (CKAD Domain 5 — 20%) + Final Assessment

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.



**Minimum 75%
attendance required**

Recap & Today

- So far: design & build (D1), deployment (D2), observability and config & security (D3).
- This morning — Domain 5 Services & Networking (20%): how Pods are reached and isolated.
- Services and DNS for stable in-cluster addressing; Ingress for HTTP/HTTPS; NetworkPolicy for isolation.
- From 1:00 PM — mock-exam practice; the final written assessment starts at 2:00 PM.



TOPIC 1

Services

Stable virtual IPs & DNS for a set of Pods

Why Services?

- Pod IPs are ephemeral — a Service gives a stable virtual IP and DNS name in front of matching Pods.
- ClusterIP (default) is in-cluster only; NodePort opens a port on every node; LoadBalancer asks the cloud for an external IP.
- A Service finds its Pods by label selector; the matching set is tracked as EndpointSlices.
- Empty Endpoints almost always means a selector/label mismatch — the most common Service bug.

Choosing a Service Type

ClusterIP

- In-cluster only
- Stable virtual IP + DNS
- Default type

NodePort

- Port on every node
- 30000–32767
- Reachable from outside

LoadBalancer

- Cloud external IP
- Builds on NodePort
- One Service, public entry

Service Types — Reaching Pods

ClusterIP

default · in-cluster only



Stable virtual IP + DNS; reachable only by other Pods.

NodePort

port 30000–32767 on every node



Opens the same port on all nodes → reachable from outside.

LoadBalancer

cloud provider LB



Provisions an external IP; builds on NodePort underneath.

Routing: selector → EndpointSlices → Pods. Empty Endpoints = selector/label mismatch (the #1 Service bug).

Services — ClusterIP, NodePort, LoadBalancer

LAB 27**Services**

A Service gives a stable virtual IP and DNS name to a set of Pods. CKAD requires fluency with ClusterIP, NodePort and LoadBalancer types, kubectl expose, endpoint debugging and the selector-mismatch pattern.

You'll build

Expose a Deployment as ClusterIP, NodePort and LoadBalancer, inspect EndpointSlices, then debug a selector mismatch.

Key commands: `k create · k expose · k patch · » ClusterIP`

Services — ClusterIP, NodePort, LoadBalancer

Step 1 — Backend Deployment + ClusterIP Service

```
k create deployment web --image=nginx:1.25 --replicas=3
k expose deployment web --port=80 --target-port=80 --name=web-cip
k describe svc web-cip | grep -E "IP:|Endpoints:"
```

Step 2 — NodePort (a port on every node)

```
k expose deployment web --port=80 --target-port=80 --type=NodePort --name=web-np
PORT=$(k get svc web-np -o jsonpath='{.spec.ports[0].nodePort}')
curl -s http://localhost:$PORT | head -3 # 30000-32767
```

Step 3 — LoadBalancer + EndpointSlices

```
k expose deployment web --port=80 --type=LoadBalancer --name=web-lb
k get endpointslices -l kubernetes.io/service-name=web-cip
```

Services — ClusterIP, NodePort, LoadBalancer (cont.)

Step 4 — Debug a selector mismatch (the #1 Service bug)

```
k patch svc web-cip -p '{"spec":{"selector":{"app":"does-not-exist"}}}'  
k get endpoints web-cip          # empty = no Pod matches  
k patch svc web-cip -p '{"spec":{"selector":{"app":"web"}}}'
```

Note ClusterIP = in-cluster only; NodePort = a port on every node; LoadBalancer = a cloud LB. Empty Endpoints almost always means the selector does not match Pod labels. EndpointSlices (v1.21+) succeed Endpoints.

Services — ClusterIP, NodePort, LoadBalancer

✓ Test it

k describe svc web-cip lists three endpoint IPs, the NodePort serves the page on localhost:\$PORT, and a bad selector empties the Endpoints until it is fixed.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 2

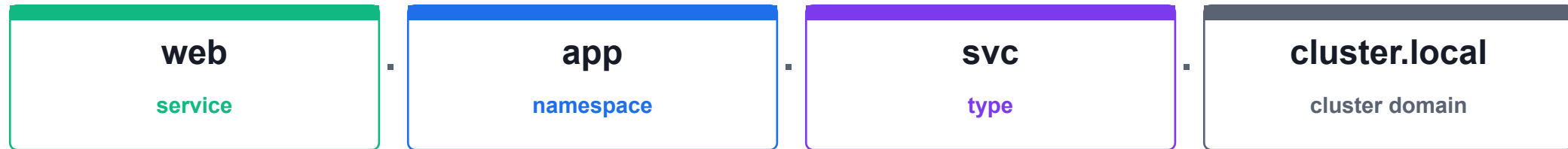
Service DNS

How CoreDNS resolves Services and Pods

Names, Not IPs

- CoreDNS gives every Service a name: `<service>.<namespace>.svc.cluster.local`.
- Within the same namespace the short name resolves via the Pod's search list.
- Cross-namespace you need at least `<service>.<namespace>`; the full FQDN always works.
- A headless Service (clusterIP: None) returns every Pod IP instead of a single virtual IP.

Service DNS Name Breakdown



- Same namespace → short name web resolves via the search list.
- Cross-namespace → use at least web.app (service.namespace).
- ndots:5 → names with <5 dots try the search list before an absolute lookup.
- Headless Service (clusterIP: None) → DNS returns every Pod IP, not one VIP.

Service DNS

LAB 28

Service DNS

CoreDNS gives every Service a stable DNS name: `<service>.<namespace>.svc.cluster.local`. CKAD tests cross-namespace resolution, Pod DNS records, headless Services, and reading `/etc/resolv.conf` to understand `ndots:5`.

You'll build

Resolve a Service by short name and FQDN, cross namespaces, read a Pod DNS record and `/etc/resolv.conf`, then a headless Service.

Key commands: `k -n nslookup web · cat /etc/resolv.conf · » FQDN`

Service DNS

Step 1 — Resolve from the same namespace (short name)

```
k -n app run client --image=busybox --restart=Never -it --rm -- \
  sh -c 'nslookup web; wget -qO- web | head -3'
```

Step 2 — Resolve from a different namespace

```
nslookup web           # fails cross-namespace
nslookup web.app       # works
nslookup web.app.svc.cluster.local
```

Step 3 — Pod DNS record and resolv.conf

```
# Pod A-record: <dashed-ip>.<namespace>.pod.cluster.local
cat /etc/resolv.conf
# search app.svc.cluster.local svc.cluster.local cluster.local
# ndots:5
```

Service DNS (cont.)

Step 4 — Headless Service returns all Pod IPs

```
k -n app create service clusterip headless --clusterip="None" --tcp=80:80
nslookup headless.app    # returns every Pod IP, not one VIP
```

Note FQDN = <service>.<namespace>.svc.cluster.local. Short names resolve via the search list within the namespace; cross-namespace needs <service>.<namespace>. ndots:5 is why short names check the search list before an absolute lookup. Headless (clusterIP: None) returns per-Pod IPs.

Service DNS

✓ Test it

nslookup web works in-namespace but needs web.app cross-namespace, and the headless Service resolves to all backing Pod IPs.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

Tea Break

15 minutes



TOPIC 3

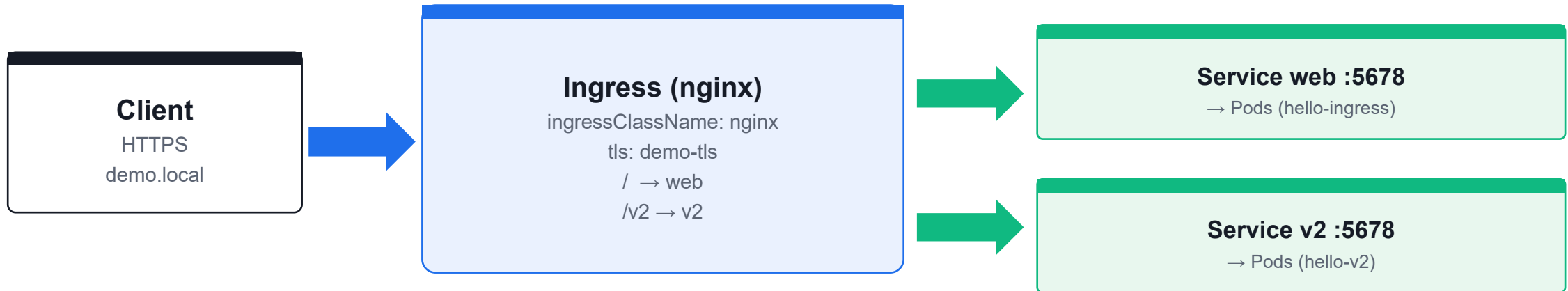
Ingress & TLS

Layer-7 HTTP/HTTPS routing into the cluster

One Entry Point, Many Rules

- An Ingress defines host- and path-based HTTP/HTTPS rules that forward to backend Services.
- An Ingress controller (e.g. ingress-nginx) must be running; ingressClassName selects it.
- TLS terminates at the controller — tls.secretName points at a kubernetes.io/tls Secret.
- pathType: Prefix matches a path and everything below it; Exact requires an exact match.
- apiVersion is networking.k8s.io/v1 (extensions/v1beta1 was removed in 1.22).

Ingress — L7 Routing with TLS



apiVersion **networking.k8s.io/v1**. TLS terminates at the controller via **tls.secretName**; pathType: Prefix matches a path and below.

Ingress with TLS

LAB 29

Ingress & TLS

An Ingress provides Layer-7 HTTP/HTTPS routing: hostname and path rules forwarding to backend Services. CKAD tests installing a controller, TLS Secrets, host routing and path-based routing with pathType.

You'll build

Install ingress-nginx, create a TLS Secret, write a host+TLS Ingress, test HTTPS, then add a path-based rule.

Key commands: `k create · apiVersion: networking.k8s.io/v1 · curl -k · » Ingress`

Ingress with TLS

Step 1 — Backend Service + TLS Secret

```
k create deployment web --image=hashicorp/http-echo --replicas=2 -- -text=hello-ingress
k expose deployment web --port=5678 --target-port=5678
k create secret tls demo-tls --cert=tls.crt --key=tls.key
```

Ingress with TLS (cont.)

Step 2 — Ingress with TLS and host routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: demo}
spec:
  ingressClassName: nginx
  tls:
  - hosts: [demo.local]
    secretName: demo-tls
  rules:
  - host: demo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web, port: {number: 5678}}}}
```

Ingress with TLS (cont.)

Step 3 — Test HTTPS routing

```
curl -k --resolve demo.local:$HTTPS_PORT:127.0.0.1 \  
https://demo.local:$HTTPS_PORT/      # hello-ingress
```

Step 4 — Add path-based routing

```
# add a second rule: path /v2 -> service v2:5678 (pathType: Prefix)  
curl -k --resolve demo.local:$HTTPS_PORT:127.0.0.1 \  
https://demo.local:$HTTPS_PORT/v2      # hello-v2
```

Note Ingress apiVersion is networking.k8s.io/v1. ingressClassName selects the controller; tls.secretName must point at a kubernetes.io/tls Secret; pathType: Prefix matches a path and everything below it. --resolve + -k let curl test without real DNS or a valid cert.

Ingress with TLS

✓ Test it

`curl https://demo.local:$HTTPS_PORT/` returns `hello-ingress` over TLS, and the `/v2` path returns `hello-v2` after the second rule is added.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>



TOPIC 4

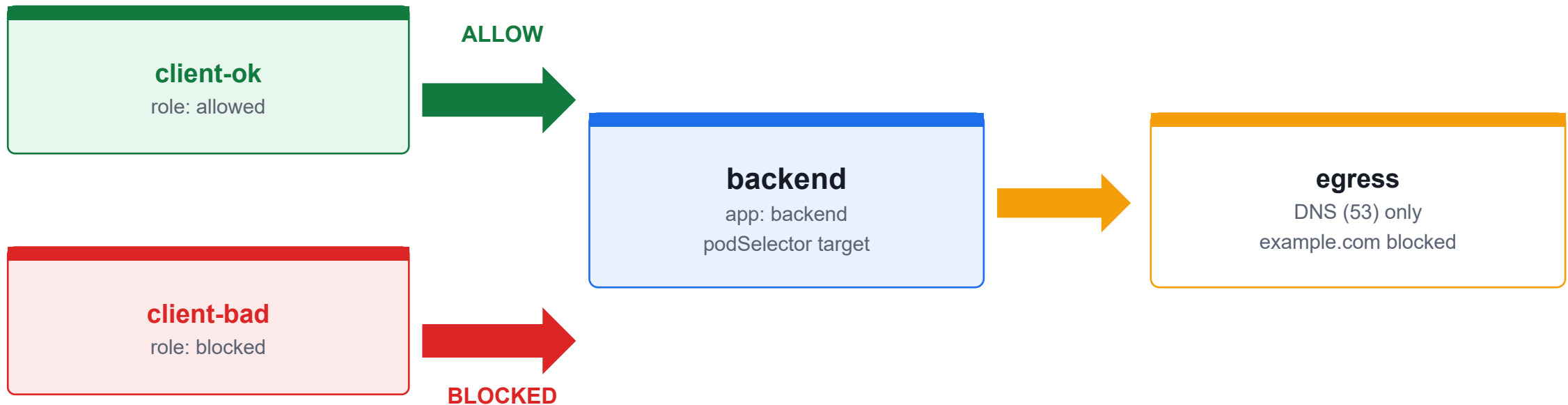
NetworkPolicy

Allow-list Pod-to-Pod traffic

Restrict the Default Open Network

- By default every Pod can talk to every other Pod — NetworkPolicies allow-list specific traffic.
- Default-deny: podSelector: {} with policyTypes:[Ingress] and no ingress rules blocks all inbound.
- Allow by podSelector (Pods by label) or namespaceSelector (whole namespaces) in from/to.
- Egress works the same way — the classic exam pattern is 'deny all egress except DNS (port 53)'.
- Policies are additive: multiple policies combine with logical OR on their allow rules.

NetworkPolicy — Allow-List Traffic



Default-deny = **podSelector: {} + policyTypes:[Ingress]**. Allow by **podSelector / namespaceSelector**. Policies are additive (OR).

NetworkPolicy

LAB 30**NetworkPolicy**

By default every Pod can talk to every other Pod. NetworkPolicies allow-list specific sources and destinations. CKAD regularly includes a NetworkPolicy question — you must write a default-deny and a selective-allow policy from scratch.

You'll build

Apply default-deny ingress, a selective podSelector allow, an egress DNS-only lockdown, then a cross-namespace allow.

Key commands: `spec: · ingress: · » Default-deny`

NetworkPolicy

Step 1 — Default deny: block all ingress

```
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

Step 2 — Selective allow: only role=allowed reaches the backend

```
spec:
  podSelector:
    matchLabels: {app: backend}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector:
        matchLabels: {role: allowed}
    ports:
    - {protocol: TCP, port: 5678}
```

NetworkPolicy (cont.)

Step 3 — Egress lockdown: allow DNS only

```
spec:
  podSelector: {matchLabels: {role: blocked}}
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector: {}
      podSelector: {matchLabels: {k8s-app: kube-dns}}
    ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
```

Step 4 — Cross-namespace allow (namespaceSelector)

```
ingress:
- from:
  - namespaceSelector:
      matchLabels: {purpose: trusted}
```

NetworkPolicy (cont.)

Note Default-deny = podSelector: {} + policyTypes: [Ingress] with no ingress list. podSelector in from matches Pods by label in the same namespace; namespaceSelector matches whole namespaces. Policies are additive — multiple policies combine with logical OR.

NetworkPolicy

✓ Test it

After the allow policy, client-ok (role=allowed) reaches the backend while client-bad stays blocked, proving the NetworkPolicy is enforced.

Killercoda: <https://killercoda.com/playgrounds/scenario/kubernetes>

COURSE COMPLETE

All five CKAD domains complete.

Design & build · Deployment · Observability · Config & Security · Services & Networking.

Lunch Break

1 hour



MOCK EXAM

Mock-Exam Practice — from 1:00 PM

Practise on [killer.sh](#) / [Killercoda](#) before the graded assessment · ask questions freely

Mock-Exam Practice (1:00 PM)

- Warm-up before the graded assessment — work hands-on questions across all five domains.
- Use `killer.sh` and `Killercoda`; rehearse the alias `k`, `$do` dry-run and `kubectl` explain workflow.
- Time yourself and flag weak areas — the real CKAD is 2 hours, 100% hands-on.
- Trainer is available for questions; the final assessment begins at 4:00 PM.



ASSESSMENT

Final Assessment — from 4:00 PM

Written / MCQ on the LMS · open book · covers all five domains and the hands-on labs

Assessment



Final Assessment

- Written / MCQ assessment on the LMS
- Mock-exam practice from 1:00 PM; final assessment from 4:00 PM
- Covers all five CKAD domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding



Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the written / MCQ assessment.

Briefing for Assessment



Clear your space

Keep only approved materials on the table.



No recording

No photos of the assessment.



Work individually

No discussion during the assessment.



Use the LMS

The assessment is delivered online.



Open book

Slides, Learner Guide & kubernetes.io are allowed.



Time's up

Submit when time is up.

THANK YOU

Thank you — and good luck with the CKAD exam!

Keep practising on `killer.sh` and `Killercoda`; the exam is 2 hours, 100% hands-on.